

短信转发助手 - 源程序鉴别材料

软件基本信息

- 软件名称: 短信转发助手
- 版本号: V2.7.2
- 著作权人: 华昊科技有限公司
- 开发者: 王士辉
- 联系邮箱: huahao@email.cn
- 开发完成日期: 2026年4月5日
- 源程序量: 约 4000 行

源程序文件清单

文件名	文件路径	行数
BootReceiver.kt	app/src/main/java/com/lanbing/smsforwarder/BootReceiver.kt	77
Constants.kt	app/src/main/java/com/lanbing/smsforwarder/Constants.kt	59
LogStore.kt	app/src/main/java/com/lanbing/smsforwarder/LogStore.kt	143
MainActivity.kt	app/src/main/java/com/lanbing/smsforwarder/MainActivity.kt	2829
models.kt	app/src/main/java/com/lanbing/smsforwarder/models.kt	30
NetworkChangeReceiver.kt	app/src/main/java/com/lanbing/smsforwarder/NetworkChangeReceiver.kt	67
SmsForegroundService.kt	app/src/main/java/com/lanbing/smsforwarder/SmsForegroundService.kt	266
SmsReceiver.kt	app/src/main/java/com/lanbing/smsforwarder/SmsReceiver.kt	604

本软件采用全部提交的方式，我们将按文件顺序提交源代码：

BootReceiver.kt

```
/*
 * 短信转发助手
 * 版本: V2.7.2
 *
 * 著作权人: 华昊科技有限公司
 * 开发者: 王士辉
 *
 * Copyright (c) 2026 华昊科技有限公司. All rights reserved.
 * 联系邮箱: huahao@email.cn
 */

package com.lanbing.smsforwarder
```

```

import android.Manifest
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.content.pm.PackageManager
import android.os.Build
import android.util.Log
import androidx.core.content.ContextCompat
import androidx.core.app.NotificationManagerCompat

class BootReceiver : BroadcastReceiver() {
    companion object {
        private const val TAG = "BootReceiver"
    }

    override fun onReceive(context: Context, intent: Intent) {
        try {
            if (intent.action != Intent.ACTION_BOOT_COMPLETED) return

            val prefs = context.getSharedPreferences(Constants.PREFS_NAME,
Context.MODE_PRIVATE)
            val startOnBoot = prefs.getBoolean(Constants.PREF_START_ON_BOOT, false)
            val enabled = prefs.getBoolean(Constants.PREF_ENABLED, false)

            if (startOnBoot && enabled) {
                // 检查通知权限
                if (!NotificationManagerCompat.from(context).areNotificationsEnabled()) {
                    Log.w(TAG, "Notification permission not granted, cannot start service on
boot")

                    LogStore.append(context, "开机启动失败：缺少通知权限")
                    return
                }

                // 检查短信权限
                val smsPermission = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
                    ContextCompat.checkSelfPermission(context,
Manifest.permission.RECEIVE_SMS) == PackageManager.PERMISSION_GRANTED
                } else {
                    true
                }

                if (!smsPermission) {
                    Log.w(TAG, "SMS permission not granted, cannot start service on boot")
                    LogStore.append(context, "开机启动失败：缺少短信权限")
                    return
                }

                try {
                    val svcIntent = Intent(context, SmsForegroundService::class.java)
                    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                        ContextCompat.startForegroundService(context, svcIntent)
                    } else {

```

```

        context.startService(svcIntent)
    }
    LogStore.append(context, "设备开机：根据设置已启动前台服务")
} catch (t: Throwable) {
    Log.e(TAG, "Failed to start service on boot", t)
    LogStore.append(context, "开机启动服务失败：${t.javaClass.simpleName}")
}
} else {
    Log.d(TAG, "开机未启动服务：startOnBoot=$startOnBoot enabled=$enabled")
}
} catch (t: Throwable) {
    Log.w(TAG, "onReceive failed", t)
}
}
}
}

```

Constants.kt

```

/*
 * 短信转发助手
 * 版本：v2.7.2
 *
 * 著作权人：华昊科技有限公司
 * 开发者：王士辉
 *
 * Copyright (c) 2026 华昊科技有限公司. All rights reserved.
 * 联系邮箱：huahao@email.cn
 */

package com.lanbing.smsforwarder

/**
 * Application constants.
 */
object Constants {
    // Logging
    const val LOG_FILE_NAME = "sms_forwarder_logs.txt"
    const val MAX_LOG_ENTRIES = 200
    const val MAX_LOG_LINE_LENGTH = 2000

    // Deduplication
    const val DUPLICATE_WINDOW_MS = 5000L

    // Retry
    const val MAX_RETRY_ATTEMPTS = 3
    const val INITIAL_RETRY_BACKOFF_MS = 2000L
    const val MAX_FAILED_MESSAGES = 100
    const val FAILED_MESSAGES_FILE = "failed_messages.json"

    // Threading
    const val THREAD_POOL_SIZE = 4
    const val BROADCAST_TIMEOUT_SECONDS = 45L

```

```

// Notification
const val NOTIFICATION_UPDATE_THROTTLE_MS = 1000L
const val NOTIFICATION_CHANNEL_ID = "sms_forwarder_channel"
const val NOTIFICATION_CHANNEL_NAME = "短信转发服务"
const val NOTIFICATION_ID = 1423

// Preference keys
const val PREFS_NAME = "app_config"
const val PREF_ENABLED = "enabled"
const val PREF_START_ON_BOOT = "start_on_boot"
const val PREF_CHANNELS = "channels"
const val PREF_KEYWORD_CONFIGS = "keyword_configs"
const val PREF_SHOW_RECEIVER_PHONE = "show_receiver_phone"
const val PREF_SHOW_SENDER_PHONE = "show_sender_phone"
const val PREF_HIGHLIGHT_VERIFICATION_CODE = "highlight_verification_code"
const val PREF_CUSTOM_SIM1_PHONE = "custom_sim1_phone"
const val PREF_CUSTOM_SIM2_PHONE = "custom_sim2_phone"

// Network
const val NETWORK_DEBOUNCE_MS = 2000L
const val CALL_TIMEOUT_SECONDS = 20L
const val CONNECT_TIMEOUT_SECONDS = 10L
const val READ_TIMEOUT_SECONDS = 20L
}

```

LogStore.kt

```

/*
 * 短信转发助手
 * 版本: v2.7.2
 *
 * 著作权人: 华昊科技有限公司
 * 开发者: 王士辉
 *
 * Copyright (c) 2026 华昊科技有限公司. All rights reserved.
 * 联系邮箱: huahao@email.cn
 */

package com.lanbing.smsforwarder

import android.content.Context
import java.io.BufferedReader
import java.io.BufferedWriter
import java.io.File
import java.io.FileInputStream
import java.io.FileOutputStream
import java.io.InputStreamReader
import java.io.OutputStreamWriter
import java.text.SimpleDateFormat
import java.util.*

```

```

object LogStore {
    private val sdf = SimpleDateFormat("yyyy-MM-dd HH:mm:ss", Locale.getDefault())
    private val lock = Any()

    private fun logFile(context: Context): File {
        val dir = context.filesDir
        if (!dir.exists()) dir.mkdirs()
        return File(dir, Constants.LOG_FILE_NAME)
    }

    fun append(context: Context, text: String) {
        try {
            val file = logFile(context)
            val time = sdf.format(Date())
            val line = "[\$time] \${if (text.length > Constants.MAX_LOG_LINE_LENGTH)
text.take(Constants.MAX_LOG_LINE_LENGTH) + "...(截断)" else text}"
            synchronized(lock) {
                // 优化: 使用 RandomAccessFile 或分批读取大文件
                if (file.exists() && file.length() > 1024 * 1024) { // 如果文件超过 1MB, 分批处
理
                    appendLargeFile(file, line)
                } else {
                    appendSmallFile(file, line)
                }
            }
        } catch (t: Throwable) {
            t.printStackTrace()
        }
    }

    private fun appendSmallFile(file: File, line: String) {
        // 小文件保持原有逻辑
        val existing = if (file.exists()) file.readText() else ""
        val newContent = line + "\n" + existing
        val lines = newContent.lines().filter { it.isNotBlank() }
        val limited = if (lines.size > Constants.MAX_LOG_ENTRIES)
lines.take(Constants.MAX_LOG_ENTRIES) else lines
        file.writeText(limited.joinToString("\n"))
    }

    private fun appendLargeFile(file: File, line: String) {
        // 大文件优化: 只读取前 N 行, 避免加载整个文件
        val tempFile = File(file.parentFile, "${file.name}.tmp")
        try {
            BufferedWriter(OutputStreamWriter(FileOutputStream(tempFile), "UTF-8")).use {
writer ->
                writer.write(line)
                writer.newLine()

                // 读取现有文件的前 MAX_ENTRIES - 1 行
                var count = 0
                BufferedReader(InputStreamReader(FileInputStream(file), "UTF-8")).use {
reader ->

```

```

        var currentLine: String?
        while (reader.readLine().also { currentLine = it } != null && count <
Constants.MAX_LOG_ENTRIES - 1) {
            if (currentLine!!.isNotBlank()) {
                writer.write(currentLine!!)
                writer.newLine()
                count++
            }
        }
    }
}

// 原子性替换
if (tempFile.exists() && file.delete()) {
    tempFile.renameTo(file)
}
} finally {
    if (tempFile.exists()) tempFile.delete()
}
}

fun readAll(context: Context): List<String> {
    try {
        val file = logFile(context)
        if (!file.exists()) return emptyList()
        synchronized(lock) {
            val lines = mutableListOf<String>()
            BufferedReader(InputStreamReader(FileInputStream(file), "UTF-8")).use { br -
>
                var line: String? = br.readLine()
                while (line != null) {
                    if (line.isNotBlank()) lines.add(line)
                    line = br.readLine()
                }
            }
            return lines
        }
    } catch (t: Throwable) {
        t.printStackTrace()
        return emptyList()
    }
}

fun clear(context: Context) {
    try {
        val file = logFile(context)
        synchronized(lock) {
            if (file.exists()) file.writeText("")
        }
    } catch (t: Throwable) {
        t.printStackTrace()
    }
}
}

```

```

fun latest(context: Context): String {
    try {
        val file = logFile(context)
        if (!file.exists()) return "暂无日志"
        synchronized(lock) {
            BufferedReader(InputStreamReader(FileInputStream(file), "UTF-8")).use { br -
>
                var line: String?
                while (br.readLine().also { line = it } != null) {
                    if (line!!.isNotBlank()) {
                        return line!!
                    }
                }
            }
        }
    } catch (t: Throwable) {
        t.printStackTrace()
    }
    return "暂无日志"
}
}

```

MainActivity.kt

```

/*
 * 短信转发助手
 * 版本: v2.7.2
 *
 * 著作权人: 华昊科技有限公司
 * 开发者: 王士辉
 *
 * Copyright (c) 2026 华昊科技有限公司. All rights reserved.
 * 联系邮箱: huahao@email.cn
 */

package com.lanbing.smsforwarder

import android.Manifest
import android.app.Activity
import android.content.Context
import android.content.Intent
import android.content.pm.PackageManager
import android.graphics.Color as AndroidColor
import android.net.Uri
import android.os.Build
import android.os.Bundle
import android.os.PowerManager
import android.provider.Settings
import android.telephony.SubscriptionManager
import android.telephony.TelephonyManager
import android.widget.Toast
import androidx.activity.ComponentActivity

```

```

import androidx.activity.compose.setContent
import androidx.activity.result.ActivityResultLauncher
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.animation.*
import androidx.compose.animation.core.*
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape

import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.*
import androidx.compose.material.icons.outlined.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.scale
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.window.Dialog
import androidx.core.app.NotificationManagerCompat
import androidx.core.content.ContextCompat
import androidx.core.view.WindowCompat
import androidx.core.view.WindowInsetsControllerCompat

import org.json.JSONArray
import org.json.JSONObject
import java.util.*

/**
 * MainActivity: 现代简洁风格的短信转发助手主界面
 * - Material Design 3 设计
 * - 支持规则测试功能
 * - 添加电量优化白名单引导
 * - 更现代的UI设计
 */

class MainActivity : ComponentActivity() {

    private lateinit var requestSmsPermissionLauncher: ActivityResultLauncher<String>

```



```

private lateinit var requestNotifPermissionLauncher: ActivityResultLauncher<String>

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    val prefs = getSharedPreferences(Constants.PREFS_NAME, Context.MODE_PRIVATE)

    requestSmsPermissionLauncher =
registerForActivityResult(ActivityResultContracts.RequestPermission()) { granted ->
    if (granted) Toast.makeText(this, "短信权限已授权", Toast.LENGTH_SHORT).show()
    else Toast.makeText(this, "请授予短信权限以接收短信", Toast.LENGTH_LONG).show()
}

    requestNotifPermissionLauncher =
registerForActivityResult(ActivityResultContracts.RequestPermission()) { granted ->
    if (granted) Toast.makeText(this, "通知权限已授权", Toast.LENGTH_SHORT).show()
    else Toast.makeText(this, "请允许通知权限以显示常驻通知", Toast.LENGTH_LONG).show()
}

    setContent {
        val isDarkTheme = isSystemInDarkTheme()
        val colorScheme = if (isDarkTheme) darkColorScheme() else lightColorScheme()

        MaterialTheme(
            colorScheme = colorScheme,
            typography = Typography()
        ) {
            val activity = LocalContext.current as Activity
            SideEffect {
                try {
                    WindowCompat.setDecorFitsSystemWindows(activity.window, false)
                    activity.window.statusBarColor = AndroidColor.TRANSPARENT
                    val controller = WindowInsetsControllerCompat(activity.window,
activity.window.decorView)
                    controller.isAppearanceLightStatusBars = !isDarkTheme
                    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                        controller.isAppearanceLightNavigationBars = !isDarkTheme
                    }
                } catch (_: Throwable) {}
            }

            SmsForwarderApp(
                onRequestSmsPermission = {
requestSmsPermissionLauncher.launch(Manifest.permission.RECEIVE_SMS) },
                onRequestNotificationPermission = {
                    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
requestNotifPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS)
                    }
                },
                onStartService = { startServiceWithNotificationCheck() },
                onStopService = { onStopService() }
            )
        }
    }
}

```

```

    }
}

private fun onStopService() {
    val svc = Intent(this, SmsForegroundService::class.java)
    stopService(svc)
}

private fun startServiceWithNotificationCheck() {
    val pkg = packageName
    val notifEnabled = NotificationManagerCompat.from(this).areNotificationsEnabled()
    if (!notifEnabled) {
        Toast.makeText(this, "请允许应用通知（将打开通知设置）", Toast.LENGTH_LONG).show()
        val i = Intent().apply {
            action = Settings.ACTION_APP_NOTIFICATION_SETTINGS
            putExtra(Settings.EXTRA_APP_PACKAGE, pkg)
        }
        startActivity(i)
        return
    }

    val smsGranted = ContextCompat.checkSelfPermission(this,
Manifest.permission.RECEIVE_SMS) == PackageManager.PERMISSION_GRANTED
    if (!smsGranted) return

    val svc = Intent(this, SmsForegroundService::class.java)
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) startForegroundService(svc) else
startService(svc)
}

fun isValidWebhookUrl(url: String): Boolean {
    return try {
        val u = java.net.URL(url)
        (u.protocol == "http" || u.protocol == "https") && u.host.isNotBlank()
    } catch (e: java.net.MalformedURLException) {
        false
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun SmsForwarderApp(
    onRequestSmsPermission: () -> Unit,
    onRequestNotificationPermission: () -> Unit,
    onStartService: () -> Unit,
    onStopService: () -> Unit
) {
    val context = LocalContext.current
    val prefs = context.getSharedPreferences(Constants.PREFS_NAME, Context.MODE_PRIVATE)

    var isEnabled by remember { mutableStateOf(prefs.getBoolean(Constants.PREF_ENABLED,
false)) }

```

```

    var startOnBoot by remember {
mutableStateOf(prefs.getBoolean(Constants.PREF_START_ON_BOOT, false)) }
    var showReceiverPhone by remember {
mutableStateOf(prefs.getBoolean(Constants.PREF_SHOW_RECEIVER_PHONE, true)) }
    var showSenderPhone by remember {
mutableStateOf(prefs.getBoolean(Constants.PREF_SHOW_SENDER_PHONE, true)) }
    var highlightVerificationCode by remember {
mutableStateOf(prefs.getBoolean(Constants.PREF_HIGHLIGHT_VERIFICATION_CODE, true)) }

    var channels by remember { mutableStateOf(loadChannels(prefs)) }
    var configs by remember { mutableStateOf(loadConfigs(prefs)) }

    // Channel form state
    var newChannelName by remember { mutableStateOf("") }
    var newChannelTarget by remember { mutableStateOf("") }
    var newChannelType by remember { mutableStateOf(ChannelType.WECHAT) }
    var channelTypeExpanded by remember { mutableStateOf(false) }

    // Config form state
    var newKeywordInput by remember { mutableStateOf("") }
    var selectedChannelIdForNewCfg by remember { mutableStateOf(channels.firstOrNull()?.id
?: "") }
    var configChannelDropdownExpanded by remember { mutableStateOf(false) }

    // Editing state
    var editingChannel by remember { mutableStateOf<Channel?>(null) }
    var showChannelDialog by remember { mutableStateOf(false) }
    var editChannelName by remember { mutableStateOf("") }
    var editChannelTarget by remember { mutableStateOf("") }
    var editChannelType by remember { mutableStateOf(ChannelType.WECHAT) }

    var editingConfig by remember { mutableStateOf<KeywordConfig?>(null) }
    var showConfigDialog by remember { mutableStateOf(false) }
    var editConfigKeyword by remember { mutableStateOf("") }
    var editConfigChannelId by remember { mutableStateOf("") }

    // UI state
    var logs by remember { mutableStateOf(LogStore.readAll(context)) }
    var currentTab by remember { mutableStateOf<Int>(0) }
    var showTestDialog by remember { mutableStateOf(false) }
    var showAboutDialog by remember { mutableStateOf(false) }

    // 定义5个标签页
    val tabs = listOf(
        "首页" to Icons.Default.Home,
        "关键词" to Icons.Default.Label,
        "通道" to Icons.Default.Cloud,
        "设置" to Icons.Default.Settings,
        "日志" to Icons.Default.History
    )

    // Permission states

```

```

        val smsGranted = ContextCompat.checkSelfPermission(context,
Manifest.permission.RECEIVE_SMS) == PackageManager.PERMISSION_GRANTED
        val notifGranted = NotificationManagerCompat.from(context).areNotificationsEnabled()

        // Battery optimization state
        val powerManager = context.getSystemService(Context.POWER_SERVICE) as PowerManager
        val isIgnoringBatteryOptimizations = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M)
{
            powerManager.isIgnoringBatteryOptimizations(context.packageName)
        } else true

        // Page indicator colors
        val pageColors = listOf(
            Color(0xFF667EEA), // Purple
            Color(0xFF10B981) // Green
        )

        Scaffold(
            modifier = Modifier.background(
                brush = Brush.verticalGradient(
                    colors = listOf(
                        MaterialTheme.colorsScheme.surface,
                        MaterialTheme.colorsScheme.surfacevariant.copy(alpha = 0.3f)
                    )
                )
            ),
            topBar = {
                TopAppBar(
                    title = {
                        Row(verticalAlignment = Alignment.CenterVertically) {
                            Box(
                                modifier = Modifier
                                    .size(40.dp)
                                    .clip(RoundedCornersShape(12.dp))
                                    .background(
                                        brush = Brush.linearGradient(
                                            colors = listOf(Color(0xFF667EEA),
Color(0xFF764BA2))
                                        )
                                    )
                            ),
                            contentAlignment = Alignment.Center
                        ) {
                            Icon(
                                imageVector = Icons.Filled.Message,
                                contentDescription = null,
                                tint = Color.white,
                                modifier = Modifier.size(24.dp)
                            )
                        }
                        Spacer(modifier = Modifier.width(12.dp))
                        Column {
                            Text(
                                "短信转发助手",

```

```

        fontSize = 20.sp,
        fontWeight = FontWeight.Bold
    )
    Text(
        if (isEnabled) "服务运行中" else "服务已停止",
        fontSize = 12.sp,
        color = if (isEnabled) Color(0xFF10B981) else
MaterialTheme.colorsScheme.onSurfaceVariant
    )
    }
    },
    actions = {
        IconButton(onClick = { showAboutDialog = true }) {
            Icon(Icons.Outlined.Info, contentDescription = "关于")
        }
    },
    colors = TopAppBarDefaults.topAppBarColors(
        containerColor = MaterialTheme.colorsScheme.surface.copy(alpha = 0.8f)
    )
),
},
bottomBar = {
    NavigationBar(
        containerColor = MaterialTheme.colorsScheme.surface,
        tonalElevation = 8.dp
    ) {
        tabs.forEachIndexed { index, (label, icon) ->
            NavigationBarItem(
                selected = currentTab == index,
                onClick = { currentTab = index },
                icon = {
                    val filledIcon = when (index) {
                        0 -> Icons.Filled.Home
                        1 -> Icons.Filled.Label
                        2 -> Icons.Filled.Cloud
                        3 -> Icons.Filled.Settings
                        4 -> Icons.Filled.History
                        else -> Icons.Filled.Home
                    }
                    val outlinedIcon = when (index) {
                        0 -> Icons.Outlined.Home
                        1 -> Icons.Outlined.Label
                        2 -> Icons.Outlined.Cloud
                        3 -> Icons.Outlined.Settings
                        4 -> Icons.Outlined.History
                        else -> Icons.Outlined.Home
                    }
                    Icon(
                        if (currentTab == index) filledIcon else outlinedIcon,
                        contentDescription = label
                    )
                }
            ),
        },
    }
}

```

```

        label = { Text(label) }
    )
}
}
} { innerPadding ->
    Box(
        modifier = Modifier
            .padding(innerPadding)
            .fillMaxSize()
    ) {
        AnimatedContent(
            targetState = currentTab,
            label = "tabAnimation"
        ) { tabIndex ->
            when (tabIndex) {
                0 -> HomeTab(
                    isEnabled = isEnabled,
                    onEnabledChange = { checked ->
                        isEnabled = checked
                        prefs.edit().putBoolean(Constants.PREF_ENABLED,
isEnabled).apply()

                        if (checked) {
                            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
                                val hasNotif =
ContextCompat.checkSelfPermission(context, Manifest.permission.POST_NOTIFICATIONS) ==
PackageManager.PERMISSION_GRANTED

                                if (!hasNotif) onRequestNotificationPermission()
                            }
                            val hasSms = ContextCompat.checkSelfPermission(context,
Manifest.permission.RECEIVE_SMS) == PackageManager.PERMISSION_GRANTED
                            if (!hasSms) onRequestSmsPermission()
                            onStartService()
                            LogStore.append(context, "服务已启动（由用户开启）")
                        } else {
                            onStopService()
                            LogStore.append(context, "服务已停止（由用户关闭）")
                        }
                    }

                    context.sendBroadcast(Intent(SmsForegroundService.ACTION_UPDATE))
                },
                startOnBoot = startOnBoot,
                onStartOnBootChange = {
                    startOnBoot = it
                    prefs.edit().putBoolean(Constants.PREF_START_ON_BOOT,
startOnBoot).apply()

                    if (startOnBoot) LogStore.append(context, "已开启开机启动") else
LogStore.append(context, "已关闭开机启动")
                },
                smsGranted = smsGranted,
                notifGranted = notifGranted,
                isIgnoringBatteryOptimizations = isIgnoringBatteryOptimizations,
                onRequestSmsPermission = onRequestSmsPermission,

```

```

        onRequestNotificationPermission = onRequestNotificationPermission,
        onRequestBatteryOptimization = {
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
                val intent =
Intent(Settings.ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS).apply {
                    data = Uri.parse("package:${context.packageName}")
                }
                context.startActivity(intent)
            }
        }
    )
    1 -> KeywordTab(
        channels = channels,
        configs = configs,
        newKeywordInput = newKeywordInput,
        onNewKeywordInputChange = { newKeywordInput = it },
        selectedChannelIdForNewCfg = selectedChannelIdForNewCfg,
        onSelectedChannelIdForNewCfgChange = { selectedChannelIdForNewCfg =
it },

        configChannelDropdownExpanded = configChannelDropdownExpanded,
        onConfigChannelDropdownExpandedChange = {
configChannelDropdownExpanded = it },
        onAddConfig = {
            if (channels.isEmpty()) {
                Toast.makeText(context, "请先添加通道",
Toast.LENGTH_SHORT).show()

                return@KeywordTab
            }
            if (selectedChannelIdForNewCfg.isBlank()) {
                Toast.makeText(context, "请选择通道",
Toast.LENGTH_SHORT).show()

                return@KeywordTab
            }
            val newCfg = KeywordConfig(UUID.randomUUID().toString(),
newKeywordInput.trim(), selectedChannelIdForNewCfg)
            configs = configs + newCfg
            saveConfigs(prefs, configs)
            newKeywordInput = ""
            LogStore.append(context, "添加关键词: ${newCfg.keyword} ->
${channels.find { it.id == newCfg.channelId }?.name}")
            Toast.makeText(context, "配置已添加", Toast.LENGTH_SHORT).show()
        },
        onDeleteConfig = { cfg ->
            configs = configs.filterNot { it.id == cfg.id }
            saveConfigs(prefs, configs)
        },
        onEditConfig = { cfg ->
            editingConfig = cfg
            editConfigKeyword = cfg.keyword
            editConfigChannelId = cfg.channelId
            showConfigDialog = true
        }
    )

```

```

2 -> ChannelTab(
    channels = channels,
    newChannelName = newChannelName,
    onNewChannelNameChange = { newChannelName = it },
    newChannelTarget = newChannelTarget,
    onNewChannelTargetChange = { newChannelTarget = it },
    newChannelType = newChannelType,
    onNewChannelTypeChange = { newChannelType = it },
    channelTypeExpanded = channelTypeExpanded,
    onChannelTypeExpandedChange = { channelTypeExpanded = it },
    onAddChannel = {
        if (newChannelName.isBlank() || newChannelTarget.isBlank()) {
            Toast.makeText(context, "请填写通道名称和 webhook 地址",
Toast.LENGTH_SHORT).show()
            return@ChannelTab
        }
        if (!isValidWebhookUrl(newChannelTarget)) {
            Toast.makeText(context, "webhook 地址格式无效, 请输入有效的
http:// 或 https:// 地址", Toast.LENGTH_SHORT).show()
            return@ChannelTab
        }
        val newChannel = Channel(UUID.randomUUID().toString(),
newChannelName.trim(), newChannelType, newChannelTarget.trim())
        channels = channels + newChannel
        saveChannels(prefs, channels)
        selectedChannelIdForNewCfg = channels.firstOrNull()?.id ?: ""
        newChannelName = ""
        newChannelTarget = ""
        LogStore.append(context, "添加通道: ${newChannel.name}
(${newChannel.type})")
        Toast.makeText(context, "通道已添加", Toast.LENGTH_SHORT).show()
    },
    onDeleteChannel = { ch ->
        channels = channels.filterNot { it.id == ch.id }
        saveChannels(prefs, channels)
        configs = configs.filterNot { it.channelId == ch.id }
        saveConfigs(prefs, configs)
        LogStore.append(context, "删除通道: ${ch.name}")
    },
    onEditChannel = { ch ->
        editingChannel = ch
        editChannelName = ch.name
        editChannelTarget = ch.target
        editChannelType = ch.type
        showChannelDialog = true
    }
)
3 -> SettingsTab(
    showReceiverPhone = showReceiverPhone,
    onShowReceiverPhoneChange = {
        showReceiverPhone = it
        prefs.edit().putBoolean(Constants.PREF_SHOW_RECEIVER_PHONE,
showReceiverPhone).apply()
    }
)

```



```

        if (showReceiverPhone) LogStore.append(context, "已开启显示本机号码")
    else LogStore.append(context, "已关闭显示本机号码")
    },
    showSenderPhone = showSenderPhone,
    onShowSenderPhoneChange = {
        showSenderPhone = it
        prefs.edit().putBoolean(Constants.PREF_SHOW_SENDER_PHONE,
showSenderPhone).apply()
        if (showSenderPhone) LogStore.append(context, "已开启显示发送者号码")
    else LogStore.append(context, "已关闭显示发送者号码")
    },
    highlightVerificationCode = highlightVerificationCode,
    onHighlightVerificationCodeChange = {
        highlightVerificationCode = it

        prefs.edit().putBoolean(Constants.PREF_HIGHLIGHT_VERIFICATION_CODE,
highlightVerificationCode).apply()
        if (highlightVerificationCode) LogStore.append(context, "已开启突出显示验证码")
    else LogStore.append(context, "已关闭突出显示验证码")
    },
    onShowTestDialog = { showTestDialog = true }
)
4 -> LogTab(
    logs = logs,
    onRefresh = { logs = LogStore.readAll(context) },
    onClear = {
        LogStore.clear(context)
        logs = emptyList()
        Toast.makeText(context, "日志已清除", Toast.LENGTH_SHORT).show()
    }
)
}
}
}
}

// Edit Channel Dialog
if (showChannelDialog && editingChannel != null) {
    ModernAlertDialog(
        onDismissRequest = { showChannelDialog = false; editingChannel = null },
        title = "编辑通道",
        content = {
            Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {
                OutlinedTextField(
                    value = editChannelName,
                    onValueChange = { editChannelName = it },
                    label = { Text("通道名称") },
                    modifier = Modifier.fillMaxWidth(),
                    shape = RoundedCornerShape(12.dp)
                )
                var editTypeExpanded by remember { mutableStateOf(false) }
                ExposedDropDownMenuBox(
                    expanded = editTypeExpanded,

```

```

        onExpandedChange = { editTypeExpanded = !editTypeExpanded }
    ) {
        OutlinedTextField(
            value = getChannelTypeLabel(editChannelType),
            onValueChange = {},
            readOnly = true,
            label = { Text("通道类型") },
            modifier = Modifier.menuAnchor().fillMaxWidth(),
            trailingIcon = {
ExposedDropdownMenuDefaults.TrailingIcon(editTypeExpanded) },
            shape = RoundedCornerShape(12.dp)
        )
        ExposedDropdownMenu(
            expanded = editTypeExpanded,
            onDismissRequest = { editTypeExpanded = false }
        ) {
            ChannelType.entries.forEach { t ->
                DropdownMenuItem(
                    text = { Text(getChannelTypeLabel(t)) },
                    onClick = {
                        editChannelType = t
                        editTypeExpanded = false
                    }
                )
            }
        }
    }
    OutlinedTextField(
        value = editChannelTarget,
        onValueChange = { editChannelTarget = it },
        label = { Text("webhook 地址") },
        modifier = Modifier.fillMaxWidth(),
        shape = RoundedCornerShape(12.dp)
    )
},
confirmButton = {
    Button(
        onClick = {
            val ch = editingChannel ?: return@Button
            val updated = Channel(ch.id, editChannelName.trim(),
editChannelType, editChannelTarget.trim())
            channels = channels.map { if (it.id == ch.id) updated else it }
            saveChannels(prefs, channels)
            LogStore.append(context, "编辑通道: ${updated.name}")
            showChannelDialog = false
            editingChannel = null
        },
        shape = RoundedCornerShape(12.dp)
    ) { Text("保存") }
},
dismissButton = {

```

```

        TextButton(onClick = { showChannelDialog = false; editingChannel = null }) {
Text("取消") }
        }
    )
}

// Edit Config Dialog
if (showConfigDialog && editingConfig != null) {
    ModernAlertDialog(
        onDismissRequest = { showConfigDialog = false; editingConfig = null },
        title = "编辑关键词配置",
        content = {
            Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {
                OutlinedTextField(
                    value = editConfigKeyword,
                    onValueChange = { editConfigKeyword = it },
                    label = { Text("关键词（留空表示全部）") },
                    modifier = Modifier.fillMaxWidth(),
                    shape = RoundedCornerShape(12.dp)
                )
                var editCfgExpanded by remember { mutableStateOf(false) }
                ExposedDropdownMenuBox(
                    expanded = editCfgExpanded,
                    onExpandedChange = { editCfgExpanded = !editCfgExpanded }
                ) {
                    OutlinedTextField(
                        value = channels.find { it.id == editConfigChannelId }?.name ?:
"选择通道",

                        onValueChange = {},
                        readOnly = true,
                        label = { Text("转发通道") },
                        modifier = Modifier.menuAnchor().fillMaxWidth(),
                        trailingIcon = {
ExposedDropdownMenuDefaults.TrailingIcon(editCfgExpanded) },
                        shape = RoundedCornerShape(12.dp)
                    )
                    ExposedDropdownMenu(
                        expanded = editCfgExpanded,
                        onDismissRequest = { editCfgExpanded = false }
                    ) {
                        channels.forEach { ch ->
                            DropdownMenuItem(
                                text = { Text(ch.name) },
                                onClick = {
                                    editConfigChannelId = ch.id
                                    editCfgExpanded = false
                                }
                            )
                        }
                    }
                }
            }
        },
    },

```

```

        confirmButton = {
            Button(
                onClick = {
                    val cfg = editingConfig ?: return@Button
                    if (editConfigChannelId.isBlank()) {
                        Toast.makeText(context, "请选择通道", Toast.LENGTH_SHORT).show()
                        return@Button
                    }
                    val updated = keywordConfig(cfg.id, editConfigKeyword.trim(),
editConfigChannelId)

                    configs = configs.map { if (it.id == cfg.id) updated else it }
                    saveConfigs(prefs, configs)
                    LogStore.append(context, "编辑关键词: ${updated.keyword} ->
${channels.find { it.id == updated.channelId }?.name}")
                    showConfigDialog = false
                    editingConfig = null
                },
                shape = RoundedCornerShape(12.dp)
            ) { Text("保存") }
        },
        dismissButton = {
            TextButton(onClick = { showConfigDialog = false; editingConfig = null }) {
Text("取消") }
        }
    )
}

// Test Dialog
if (showTestDialog) {
    TestRuleDialog(
        channels = channels,
        configs = configs,
        onDismiss = { showTestDialog = false }
    )
}

// About Dialog
if (showAboutDialog) {
    AboutDialog(onDismiss = { showAboutDialog = false })
}
}

@Composable
fun ModernAlertDialog(
    onDismissRequest: () -> Unit,
    title: String,
    content: @Composable ColumnScope.() -> Unit,
    confirmButton: @Composable () -> Unit,
    dismissButton: @Composable () -> Unit
) {
    Dialog(onDismissRequest = onDismissRequest) {
        Card(
            modifier = Modifier

```

```

        .fillMaxWidth()
        .padding(16.dp),
        shape = RoundedCornerShape(20.dp),
        elevation = CardDefaults.cardElevation(8.dp)
    ) {
        Column(modifier = Modifier.padding(24.dp)) {
            Text(
                text = title,
                style = MaterialTheme.typography.headlineSmall,
                fontWeight = FontWeight.Bold
            )
            Spacer(modifier = Modifier.height(20.dp))
            content()
            Spacer(modifier = Modifier.height(24.dp))
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.End
            ) {
                dismissButton()
                Spacer(modifier = Modifier.width(8.dp))
                confirmButton()
            }
        }
    }
}
}
}
}
}

```

@Composable

```

fun ConfigTab(
    isEnabled: Boolean,
    onEnabledChange: (Boolean) -> Unit,
    startOnBoot: Boolean,
    onStartOnBootChange: (Boolean) -> Unit,
    showReceiverPhone: Boolean,
    onShowReceiverPhoneChange: (Boolean) -> Unit,
    showSenderPhone: Boolean,
    onShowSenderPhoneChange: (Boolean) -> Unit,
    highlightVerificationCode: Boolean,
    onHighlightVerificationCodeChange: (Boolean) -> Unit,
    smsGranted: Boolean,
    notifGranted: Boolean,
    isIgnoringBatteryOptimizations: Boolean,
    onRequestSmsPermission: () -> Unit,
    onRequestNotificationPermission: () -> Unit,
    onRequestBatteryOptimization: () -> Unit,
    channels: List<Channel>,
    configs: List<KeywordConfig>,
    newChannelName: String,
    onNewChannelNameChange: (String) -> Unit,
    newChannelTarget: String,
    onNewChannelTargetChange: (String) -> Unit,
    newChannelType: ChannelType,
    onNewChannelTypeChange: (ChannelType) -> Unit,

```

```

channelTypeExpanded: Boolean,
onChannelTypeExpandedChange: (Boolean) -> Unit,
onAddChannel: () -> Unit,
onDeleteChannel: (Channel) -> Unit,
onEditChannel: (Channel) -> Unit,
newKeywordInput: String,
onNewKeywordInputChange: (String) -> Unit,
selectedChannelIdForNewCfg: String,
onSelectedChannelIdForNewCfgChange: (String) -> Unit,
configChannelDropdownExpanded: Boolean,
onConfigChannelDropdownExpandedChange: (Boolean) -> Unit,
onAddConfig: () -> Unit,
onDeleteConfig: (KeywordConfig) -> Unit,
onEditConfig: (KeywordConfig) -> Unit
) {
    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding(horizontal = 16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp),
        contentPadding = PaddingValues(vertical = 16.dp)
    ) {
        // Service Status Card
        item {
            ModernCard(
                modifier = Modifier.fillMaxWidth()
            ) {
                Column(modifier = Modifier.padding(20.dp)) {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Column(modifier = Modifier.weight(1f)) {
                            Text(
                                "转发服务",
                                style = MaterialTheme.typography.titleLarge,
                                fontWeight = FontWeight.Bold
                            )
                            Spacer(modifier = Modifier.height(6.dp))
                            Row(verticalAlignment = Alignment.CenterVertically) {
                                val statusColor = if (isEnabled) Color(0xFF10B981) else
                                Color(0xFF9CA3AF)

                                Box(
                                    modifier = Modifier
                                        .size(12.dp)
                                        .background(statusColor, shape = CircleShape)
                                )
                                Spacer(modifier = Modifier.width(8.dp))
                                Text(
                                    if (isEnabled) "服务运行中" else "服务已停止",
                                    style = MaterialTheme.typography.bodyMedium,
                                    color = MaterialTheme.colorScheme.onSurfaceVariant
                                )
                            )
                        }
                    )
                }
            }
        }
    }
}

```

```

    }
}
val switchScale by animateFloatAsState(
    targetValue = if (isEnabled) 1.1f else 1f,
    animationSpec = spring(stiffness = Spring.StiffnessMedium),
    label = "switchAnimation"
)
Switch(
    checked = isEnabled,
    onCheckedChange = onEnabledChange,
    modifier = Modifier.scale(switchScale),
    colors = SwitchDefaults.colors(
        checkedThumbColor = Color.White,
        checkedTrackColor = Color(0xFF667EEA)
    )
)
}

Spacer(modifier = Modifier.height(20.dp))

// Permission items
PermissionItem(
    icon = Icons.Outlined.Notifications,
    title = "通知权限",
    granted = notifGranted,
    onClick = onRequestNotificationPermission
)
Spacer(modifier = Modifier.height(12.dp))
PermissionItem(
    icon = Icons.Outlined.Sms,
    title = "短信权限",
    granted = smsGranted,
    onClick = onRequestSmsPermission
)
Spacer(modifier = Modifier.height(12.dp))
PermissionItem(
    icon = Icons.Outlined.BatteryFull,
    title = "电池优化白名单",
    granted = isIgnoringBatteryOptimizations,
    onClick = onRequestBatteryOptimization
)

Spacer(modifier = Modifier.height(20.dp))

HorizontalDivider()

Spacer(modifier = Modifier.height(16.dp))

Row(
    verticalAlignment = Alignment.CenterVertically,
    modifier = Modifier.fillMaxWidth()
) {
    Icon(

```

```

        Icons.Outlined.PowerSettingsNew,
        contentDescription = null,
        tint = MaterialTheme.colorScheme.onSurfaceVariant
    )
    Spacer(modifier = Modifier.width(12.dp))
    Column(modifier = Modifier.weight(1f)) {
        Text(
            "开机自启动",
            style = MaterialTheme.typography.bodyLarge,
            fontWeight = FontWeight.Medium
        )
        Text(
            "设备启动后自动运行",
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
    Switch(
        checked = startOnBoot,
        onCheckedChange = onStartOnBootChange
    )
}

Spacer(modifier = Modifier.height(16.dp))

HorizontalDivider()

Spacer(modifier = Modifier.height(16.dp))

// 消息格式配置
Text(
    "消息格式",
    style = MaterialTheme.typography.titleLarge,
    fontWeight = FontWeight.Bold
)

Spacer(modifier = Modifier.height(12.dp))

// 显示本机号码
Row(
    verticalAlignment = Alignment.CenterVertically,
    modifier = Modifier.fillMaxWidth()
) {
    Icon(
        Icons.Outlined.Phone,
        contentDescription = null,
        tint = MaterialTheme.colorScheme.onSurfaceVariant
    )
    Spacer(modifier = Modifier.width(12.dp))
    Column(modifier = Modifier.weight(1f)) {
        Text(
            "显示本机号码",
            style = MaterialTheme.typography.bodyLarge,

```



```

        fontweight = FontWeight.Medium
    )
    Text(
        "转发时显示接收短信的本机号码",
        style = MaterialTheme.typography.bodySmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )
}
Switch(
    checked = showReceiverPhone,
    onCheckedChange = onShowReceiverPhoneChange
)
}

Spacer(modifier = Modifier.height(12.dp))

// 显示发送者号码
Row(
    verticalAlignment = Alignment.CenterVertically,
    modifier = Modifier.fillMaxWidth()
) {
    Icon(
        Icons.Outlined.Person,
        contentDescription = null,
        tint = MaterialTheme.colorScheme.onSurfaceVariant
    )
    Spacer(modifier = Modifier.width(12.dp))
    Column(modifier = Modifier.weight(1f)) {
        Text(
            "显示发送者号码",
            style = MaterialTheme.typography.bodyLarge,
            fontweight = FontWeight.Medium
        )
        Text(
            "转发时显示短信发送者号码",
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
    Switch(
        checked = showSenderPhone,
        onCheckedChange = onShowSenderPhoneChange
    )
}

Spacer(modifier = Modifier.height(12.dp))

// 突出显示验证码
Row(
    verticalAlignment = Alignment.CenterVertically,
    modifier = Modifier.fillMaxWidth()
) {
    Icon(

```

```

        Icons.Outlined.VpnKey,
        contentDescription = null,
        tint = MaterialTheme.colorScheme.onSurfaceVariant
    )
    Spacer(modifier = Modifier.width(12.dp))
    Column(modifier = Modifier.weight(1f)) {
        Text(
            "突出显示验证码",
            style = MaterialTheme.typography.bodyLarge,
            fontWeight = FontWeight.Medium
        )
        Text(
            "自动识别并突出显示短信验证码",
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
    Switch(
        checked = highlightVerificationCode,
        onCheckedChange = onHighlightVerificationCodeChange
    )
    }
    }
}

// SIM 卡信息卡片
item {
    SimCardInfoCard()
}

// Keyword Config Card
item {
    ModernCard {
        Column(
            modifier = Modifier.padding(20.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Box(
                    modifier = Modifier
                        .size(40.dp)
                        .clip(RoundedCornerShape(12.dp))
                        .background(Color(0xFF10B981).copy(alpha = 0.1f)),
                    contentAlignment = Alignment.Center
                ) {
                    Icon(
                        imageVector = Icons.Filled.Rule,
                        contentDescription = null,
                        tint = Color(0xFF10B981)
                    )
                }
            }
            Spacer(modifier = Modifier.width(12.dp))
        }
    }
}

```

```

        Column {
            Text(
                "关键词配置",
                style = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.Bold
            )
            Text(
                "设置转发关键词",
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorsScheme.onSurfaceVariant
            )
        }
    }

    OutlinedTextField(
        value = newKeywordInput,
        onValueChange = onNewKeywordInputChange,
        label = { Text("输入关键词（留空表示全部）") },
        modifier = Modifier.fillMaxWidth(),
        leadingIcon = {
            Icon(Icons.Outlined.Search, contentDescription = null)
        },
        shape = RoundedCornerShape(12.dp)
    )

    ExposedDropdownMenuBox(
        expanded = configChannelDropdownExpanded,
        onExpandedChange = onConfigChannelDropdownExpandedChange
    ) {
        OutlinedTextField(
            value = channels.find { it.id == selectedChannelIdForNewCfg
}?.name ?: "选择转发通道",
            onValueChange = {},
            readOnly = true,
            label = { Text("转发通道") },
            modifier = Modifier.menuAnchor().fillMaxWidth(),
            leadingIcon = {
                Icon(Icons.Outlined.Send, contentDescription = null)
            },
            trailingIcon = {
ExposedDropdownMenuDefaults.TrailingIcon(configChannelDropdownExpanded) },
            shape = RoundedCornerShape(12.dp)
        )
        ExposedDropdownMenu(
            expanded = configChannelDropdownExpanded,
            onDismissRequest = {
onConfigChannelDropdownExpandedChange(false) }
        ) {
            channels.forEach { ch ->
                DropdownMenuItem(
                    text = { Text(ch.name) },
                    onClick = {
                        onSelectedChannelIdForNewCfgChange(ch.id)

```

```

        onConfigChannelDropdownExpandedChange(false)
    }
    )
}
if (channels.isEmpty()) {
    DropdownMenuItem(
        text = { Text("请先添加通道") },
        onClick = { onConfigChannelDropdownExpandedChange(false)
}

    )
}
}
}

Button(
    onClick = onAddConfig,
    modifier = Modifier.fillMaxWidth(),
    shape = RoundedCornerShape(12.dp),
    contentPadding = PaddingValues(vertical = 14.dp)
) {
    Icon(Icons.Outlined.Add, contentDescription = null)
    Spacer(modifier = Modifier.width(8.dp))
    Text("添加配置", fontSize = 16.sp)
}

// Config list
if (configs.isNotEmpty()) {
    Spacer(modifier = Modifier.height(8.dp))
    configs.forEach { cfg ->
        val chName = channels.find { it.id == cfg.channelId }?.name ?: "
(已删除通道)"

        ConfigItem(
            keyword = cfg.keyword,
            channelName = chName,
            onEdit = { onEditConfig(cfg) },
            onDelete = { onDeleteConfig(cfg) }
        )
    }
}

}

}

}

// Channel Management Card
item {
    ModernCard {
        Column(
            modifier = Modifier.padding(20.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Box(
                    modifier = Modifier

```

```

        .size(40.dp)
        .clip(RoundedCornerShape(12.dp))
        .background(Color(0xFF667EEA).copy(alpha = 0.1f)),
        contentAlignment = Alignment.Center
    ) {
        Icon(
            imageVector = Icons.Filled.Cloud,
            contentDescription = null,
            tint = Color(0xFF667EEA)
        )
    }
    Spacer(modifier = Modifier.width(12.dp))
    Column {
        Text(
            "转发通道管理",
            style = MaterialTheme.typography.titleLarge,
            fontWeight = FontWeight.Bold
        )
        Text(
            "添加和管理转发通道",
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
}

OutlinedTextField(
    value = newChannelName,
    onValueChange = onNewChannelNameChange,
    label = { Text("通道名称") },
    modifier = Modifier.fillMaxWidth(),
    leadingIcon = {
        Icon(Icons.Outlined.Label, contentDescription = null)
    },
    shape = RoundedCornerShape(12.dp)
)

ExposedDropdownMenuBox(
    expanded = channelTypeExpanded,
    onExpandedChange = onChannelTypeExpandedChange
) {
    OutlinedTextField(
        value = getChannelTypeLabel(newChannelType),
        onValueChange = {},
        readOnly = true,
        label = { Text("通道类型") },
        modifier = Modifier.menuAnchor().fillMaxWidth(),
        leadingIcon = {
            Icon(Icons.Outlined.Category, contentDescription = null)
        },
        trailingIcon = {
            ExposedDropdownMenuDefaults.TrailingIcon(channelTypeExpanded) },
        shape = RoundedCornerShape(12.dp)
    )
}

```

```

    )
    ExposedDropdownMenu(
        expanded = channelTypeExpanded,
        onDismissRequest = { onChannelTypeExpandedChange(false) }
    ) {
        ChannelType.entries.forEach { t ->
            DropdownMenuItem(
                text = { Text(getChannelTypeLabel(t)) },
                onClick = {
                    onNewChannelTypeChange(t)
                    onChannelTypeExpandedChange(false)
                }
            )
        }
    }
}

OutlinedTextField(
    value = newChannelTarget,
    onValueChange = onNewChannelTargetChange,
    label = { Text("webhook 地址") },
    modifier = Modifier.fillMaxWidth(),
    leadingIcon = {
        Icon(Icons.Outlined.Link, contentDescription = null)
    },
    shape = RoundedCornerShape(12.dp)
)

Button(
    onClick = onAddChannel,
    modifier = Modifier.fillMaxWidth(),
    shape = RoundedCornerShape(12.dp),
    contentPadding = PaddingValues(vertical = 14.dp)
) {
    Icon(Icons.Outlined.Add, contentDescription = null)
    Spacer(modifier = Modifier.width(8.dp))
    Text("添加通道", fontSize = 16.sp)
}

// Channel list
if (channels.isNotEmpty()) {
    Spacer(modifier = Modifier.height(8.dp))
    channels.forEach { ch ->
        ChannelItem(
            name = ch.name,
            type = getChannelTypeLabel(ch.type),
            target = ch.target,
            onEdit = { onEditChannel(ch) },
            onDelete = { onDeleteChannel(ch) }
        )
    }
}
}

```

```

    }
}

item { Spacer(Modifier.height(32.dp)) }
}

@Composable
fun LogTab(
    logs: List<String>,
    onRefresh: () -> Unit,
    onClear: () -> Unit
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        ModernCard(
            modifier = Modifier.weight(1f)
        ) {
            Column(modifier = Modifier.padding(20.dp)) {
                Row(
                    verticalAlignment = Alignment.CenterVertically,
                    modifier = Modifier.fillMaxWidth()
                ) {
                    Box(
                        modifier = Modifier
                            .size(40.dp)
                            .clip(RoundedCornerRadius(12.dp))
                            .background(Color(0xFFFF59E0B).copy(alpha = 0.1f)),
                        contentAlignment = Alignment.Center
                    ) {
                        Icon(
                            imageVector = Icons.Filled.History,
                            contentDescription = null,
                            tint = Color(0xFFFF59E0B)
                        )
                    }
                    Spacer(modifier = Modifier.width(12.dp))
                    Column(modifier = Modifier.weight(1f)) {
                        Text(
                            "转发日志",
                            style = MaterialTheme.typography.titleLarge,
                            fontWeight = FontWeight.Bold
                        )
                        Text(
                            "${logs.size} 条记录",
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorsScheme.onSurfaceVariant
                        )
                    }
                    Row {

```

```

        IconButton(
            onClick = onRefresh,
            modifier = Modifier
                .size(44.dp)
                .clip(RoundedCornerShape(12.dp))
                .background(MaterialTheme.colorScheme.surfaceVariant)
        ) {
            Icon(Icons.Filled.Refresh, contentDescription = "刷新")
        }
        Spacer(modifier = Modifier.width(8.dp))
        IconButton(
            onClick = onClear,
            modifier = Modifier
                .size(44.dp)
                .clip(RoundedCornerShape(12.dp))
                .background(Color(0xFFEE4444).copy(alpha = 0.1f))
        ) {
            Icon(Icons.Filled.ClearAll, contentDescription = "清除", tint =
color(0xFFEE4444))
        }
    }
}
Spacer(modifier = Modifier.height(16.dp))
if (logs.isEmpty()) {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            Icon(
                Icons.Outlined.Description,
                contentDescription = null,
                modifier = Modifier.size(64.dp),
                tint = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha
= 0.5f)
            )
            Spacer(modifier = Modifier.height(16.dp))
            Text(
                "暂无日志",
                style = MaterialTheme.typography.bodyLarge,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
        }
    }
} else {
    LazyColumn(modifier = Modifier.fillMaxSize()) {
        items(logs) { line ->
            LogItem(line = line)
        }
    }
}
}
}

```



```

    }
}

@Composable
fun ModernCard(
    modifier: Modifier = Modifier,
    content: @Composable ColumnScope.() -> Unit
) {
    Card(
        modifier = modifier.fillMaxWidth(),
        shape = RoundedCornersShape(20.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorsScheme.surface
        ),
        elevation = CardDefaults.cardElevation(2.dp)
    ) {
        Column {
            content()
        }
    }
}

@Composable
fun PermissionItem(
    icon: ImageVector,
    title: String,
    granted: Boolean,
    onClick: () -> Unit
) {
    Row(
        verticalAlignment = Alignment.CenterVertically,
        modifier = Modifier
            .fillMaxWidth()
            .clip(RoundedCornersShape(12.dp))
            .background(if (granted) Color(0xFF10B981).copy(alpha = 0.1f) else
Color(0xFFEE4444).copy(alpha = 0.1f))
            .padding(12.dp)
    ) {
        Icon(
            icon,
            contentDescription = null,
            tint = if (granted) Color(0xFF10B981) else Color(0xFFEE4444)
        )
        Spacer(Modifier.width(12.dp))
        Column(modifier = Modifier.weight(1f)) {
            Text(
                title,
                style = MaterialTheme.typography.bodyLarge,
                fontWeight = FontWeight.Medium
            )
            Text(
                if (granted) "已授权" else "未授权",
                style = MaterialTheme.typography.bodySmall,

```

```

        color = MaterialTheme.colorsScheme.onSurfaceVariant
    )
}
if (granted) {
    Icon(
        Icons.Filled.CheckCircle,
        contentDescription = "已授权",
        tint = Color(0xFF10B981)
    )
} else {
    Button(
        onClick = onClick,
        shape = RoundedCornerShape(8.dp),
        contentPadding = PaddingValues(horizontal = 16.dp, vertical = 8.dp)
    ) {
        Text("去开启")
    }
}
}
}

@Composable
fun ConfigItem(
    keyword: String,
    channelName: String,
    onEdit: () -> Unit,
    onDelete: () -> Unit
) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 4.dp),
        shape = RoundedCornerShape(12.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorsScheme.surfaceVariant.copy(alpha = 0.5f)
        )
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Box(
                modifier = Modifier
                    .size(40.dp)
                    .clip(CircleShape)
                    .background(Color(0xFF10B981).copy(alpha = 0.2f)),
                contentAlignment = Alignment.Center
            ) {
                Icon(
                    Icons.Filled.Rule,
                    contentDescription = null,

```

```

        tint = Color(0xFF10B981),
        modifier = Modifier.size(20.dp)
    )
}
Spacer(modifier = Modifier.width(12.dp))
Column(modifier = Modifier.weight(1f)) {
    Text(
        if (keyword.isBlank()) "全部消息" else keyword,
        style = MaterialTheme.typography.bodyLarge,
        fontWeight = FontWeight.Medium
    )
    Text(
        "→ $channelName",
        style = MaterialTheme.typography.bodySmall,
        color = MaterialTheme.colorsScheme.onSurfaceVariant
    )
}
IconButton(
    onClick = onEdit,
    modifier = Modifier.size(40.dp)
) {
    Icon(Icons.Filled.Edit, contentDescription = "编辑")
}
IconButton(
    onClick = onDelete,
    modifier = Modifier.size(40.dp)
) {
    Icon(Icons.Filled.Delete, contentDescription = "删除", tint =
Color(0xFFEE4444))
}
}
}
}
}

```

@Composable

```

fun ChannelItem(
    name: String,
    type: String,
    target: String,
    onEdit: () -> Unit,
    onDelete: () -> Unit
) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 4.dp),
        shape = RoundedCornerShape(12.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorsScheme.surfaceVariant.copy(alpha = 0.5f)
        )
    ) {
        Row(
            modifier = Modifier

```

```

        .fillMaxWidth()
        .padding(16.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        val channelColor = when (type) {
            "企业微信" -> color(0xFF07C160)
            "钉钉" -> color(0xFF2080F0)
            "飞书" -> color(0xFF2064E5)
            else -> color(0xFF667EEA)
        }
        Box(
            modifier = Modifier
                .size(40.dp)
                .clip(CircleShape)
                .background(channelColor.copy(alpha = 0.2f)),
            contentAlignment = Alignment.Center
        ) {
            Icon(
                Icons.Filled.Cloud,
                contentDescription = null,
                tint = channelColor,
                modifier = Modifier.size(20.dp)
            )
        }
        Spacer(modifier = Modifier.width(12.dp))
        Column(modifier = Modifier.weight(1f)) {
            Text(
                name,
                style = MaterialTheme.typography.bodyLarge,
                fontWeight = FontWeight.Medium
            )
            Text(
                "$type -> ${target.take(30)}${if (target.length > 30) "... " else ""}",
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorsScheme.onSurfaceVariant,
                maxLines = 1,
                overflow = TextOverflow.Ellipsis
            )
        }
        IconButton(
            onClick = onEdit,
            modifier = Modifier.size(40.dp)
        ) {
            Icon(Icons.Filled.Edit, contentDescription = "编辑")
        }
        IconButton(
            onClick = onDelete,
            modifier = Modifier.size(40.dp)
        ) {
            Icon(Icons.Filled.Delete, contentDescription = "删除", tint =
color(0xFFEE4444))
        }
    }
}

```

```

    }
}

@Composable
fun LogItem(line: String) {
    val tsRegex = """"^[(.*?)\\s*(.*)$"""".toRegex()
    val match = tsRegex.find(line)
    val time = match?.groups?.get(1)?.value ?: ""
    val msg = match?.groups?.get(2)?.value ?: line
    val isSuccess = msg.contains("成功") || msg.contains("已启动")
    val isError = msg.contains("失败") || msg.contains("异常") || msg.contains("错误")
    val iconTint = when {
        isSuccess -> Color(0xFF10B981)
        isError -> Color(0xFFEE4444)
        else -> MaterialTheme.colorScheme.primary
    }

    val bgColor = when {
        isSuccess -> Color(0xFF10B981).copy(alpha = 0.05f)
        isError -> Color(0xFFEE4444).copy(alpha = 0.05f)
        else -> Color.Transparent
    }

    Row(
        modifier = Modifier
            .fillMaxWidth()
            .clip(RoundedCornerRadius(8.dp))
            .background(bgColor)
            .padding(vertical = 12.dp, horizontal = 4.dp),
        verticalAlignment = Alignment.Top
    ) {
        Box(
            modifier = Modifier
                .size(8.dp)
                .background(iconTint, shape = CircleShape)
                .padding(top = 6.dp)
        )
        Spacer(Modifier.width(12.dp))
        Column(modifier = Modifier.weight(1f)) {
            Text(
                msg,
                style = MaterialTheme.typography.bodyMedium
            )
            if (time.isNotEmpty()) {
                Spacer(modifier = Modifier.height(4.dp))
                Text(
                    time,
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
            }
        }
    }
}
}

```

```

@Composable
fun TestRuleDialog(
    channels: List<Channel>,
    configs: List<KeywordConfig>,
    onDismiss: () -> Unit
) {
    var testContent by remember { mutableStateOf("") }
    var testResults by remember { mutableStateOf<List<Pair<Channel, KeywordConfig>>>
(emptyList()) }

    Dialog(onDismissRequest = onDismiss) {
        Card(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp),
            shape = RoundedCornerShape(24.dp),
            elevation = CardDefaults.cardElevation(8.dp)
        ) {
            Column(modifier = Modifier.padding(24.dp)) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Box(
                        modifier = Modifier
                            .size(48.dp)
                            .clip(RoundedCornerShape(16.dp))
                            .background(
                                brush = Brush.linearGradient(
                                    colors = listOf(Color(0xFFFF59E0B), Color(0xFFD97706))
                                )
                            ),
                        contentAlignment = Alignment.Center
                    ) {
                        Icon(
                            Icons.Filled.Science,
                            contentDescription = null,
                            tint = Color.White,
                            modifier = Modifier.size(28.dp)
                        )
                    }
                    Spacer(modifier = Modifier.width(16.dp))
                    Column {
                        Text(
                            "测试转发规则",
                            style = MaterialTheme.typography.headlineSmall,
                            fontWeight = FontWeight.Bold
                        )
                        Text(
                            "输入短信内容测试匹配",
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorsScheme.onSurfaceVariant
                        )
                    }
                }
            }
        }
    }
}

```

```

        Spacer(modifier = Modifier.height(24.dp))

        OutlinedTextField(
            value = testContent,
            onChange = {
                testContent = it
                testResults = if (it.isNotBlank()) {
                    configs.mapNotNull { cfg ->
                        val kw = cfg.keyword.trim()
                        val match = if (kw.isEmpty()) true else it.contains(kw,
ignoreCase = true)

                        if (match) {
                            channels.find { ch -> ch.id == cfg.channelId }?.let { ch
->

                                ch to cfg
                            }
                        } else null
                    }
                } else emptyList()
            },
            label = { Text("输入测试短信内容") },
            modifier = Modifier.fillMaxWidth(),
            minLines = 4,
            shape = RoundedCornerShape(16.dp)
        )

        Spacer(modifier = Modifier.height(20.dp))

        Text(
            "匹配结果",
            style = MaterialTheme.typography.titleLarge,
            fontWeight = FontWeight.SemiBold
        )

        Spacer(modifier = Modifier.height(12.dp))

        if (testResults.isEmpty()) {
            Card(
                modifier = Modifier.fillMaxWidth(),
                colors = CardDefaults.cardColors(
                    containerColor =
MaterialTheme.colorsScheme.surfaceVariant.copy(alpha = 0.5f)
                ),
                shape = RoundedCornerShape(16.dp)
            ) {
                Box(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(24.dp),
                    contentAlignment = Alignment.Center
                ) {
                    Column(horizontalAlignment = Alignment.CenterHorizontally) {
                        Icon(
                            Icons.Outlined.Rule,

```

```

        contentDescription = null,
        modifier = Modifier.size(48.dp),
        tint =
MaterialTheme.colorsScheme.onSurfaceVariant.copy(alpha = 0.5f)
    )
    Spacer(modifier = Modifier.height(12.dp))
    Text(
        if (testContent.isBlank()) "请输入内容开始测试" else "没有匹
配的规则",

        style = MaterialTheme.typography.bodyLarge,
        color = MaterialTheme.colorsScheme.onSurfaceVariant,
        textAlign = TextAlign.Center
    )
    }
}
} else {
    Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {
        testResults.forEach { (ch, cfg) ->
            Card(
                modifier = Modifier.fillMaxWidth(),
                colors = CardDefaults.cardColors(
                    containerColor = Color(0xFF10B981).copy(alpha = 0.1f)
                ),
                shape = RoundedCornerShape(16.dp)
            ) {
                Row(
                    modifier = Modifier.padding(16.dp),
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Box(
                        modifier = Modifier
                            .size(40.dp)
                            .clip(CircleShape)
                            .background(Color(0xFF10B981).copy(alpha =
0.2f)),

                        contentAlignment = Alignment.Center
                    ) {
                        Icon(
                            Icons.Filled.CheckCircle,
                            contentDescription = null,
                            tint = Color(0xFF10B981),
                            modifier = Modifier.size(24.dp)
                        )
                    }
                    Spacer(modifier = Modifier.width(12.dp))
                    Column(modifier = Modifier.weight(1f)) {
                        Text(
                            "关键词: ${if (cfg.keyword.isBlank()) "全部" else
cfg.keyword}",

                            style = MaterialTheme.typography.bodyLarge,
                            fontWeight = FontWeight.Medium
                        )
                    }
                }
            }
        }
    }
}

```



```

        Icons.Filled.Message,
        contentDescription = null,
        tint = Color.White,
        modifier = Modifier.size(40.dp)
    )
}
Spacer(modifier = Modifier.height(20.dp))
Text(
    "短信转发助手",
    style = MaterialTheme.typography.headlineSmall,
    fontWeight = FontWeight.Bold
)
Text(
    "版本 2.7.2",
    style = MaterialTheme.typography.bodyMedium,
    color = MaterialTheme.colorScheme.onSurfaceVariant
)
Spacer(modifier = Modifier.height(20.dp))
Text(
    "轻量、稳定、开源的 Android 短信转发应用。支持企业微信、钉钉、飞书和自定义 webhook 等多种转发渠道，支持关键词过滤、验证码提取、本机号码识别等功能。",
    style = MaterialTheme.typography.bodyMedium,
    textAlign = TextAlign.Center,
    color = MaterialTheme.colorScheme.onSurfaceVariant
)
Spacer(modifier = Modifier.height(16.dp))
HorizontalDivider()
Spacer(modifier = Modifier.height(16.dp))
Text(
    "© 2026 华昊科技有限公司",
    style = MaterialTheme.typography.bodySmall,
    color = MaterialTheme.colorScheme.onSurfaceVariant,
    textAlign = TextAlign.Center
)
Spacer(modifier = Modifier.height(20.dp))

Button(
    onClick = onDismiss,
    modifier = Modifier.fillMaxWidth(),
    shape = RoundedCornerShape(12.dp),
    contentPadding = PaddingValues(vertical = 14.dp)
) { Text("关闭", fontSize = 16.sp) }
}
}
}

@Composable
fun SimCardInfoCard() {
    val context = LocalContext.current
    val prefs = remember { context.getSharedPreferences(Constants.PREFS_NAME, Context.MODE_PRIVATE) }

```

```

        var customSim1Phone by remember {
mutableStateOf(prefs.getString(Constants.PREF_CUSTOM_SIM1_PHONE, null)) }
        var customSim2Phone by remember {
mutableStateOf(prefs.getString(Constants.PREF_CUSTOM_SIM2_PHONE, null)) }

        val simCards = remember(customSim1Phone, customSim2Phone) {
            getSimCardInfo(context, customSim1Phone, customSim2Phone)
        }

        var showEditDialog by remember { mutableStateOf<Int?>(null) }
        var editPhoneNumber by remember { mutableStateOf("") }

ModernCard {
    Column(modifier = Modifier.padding(20.dp)) {
        Row(verticalAlignment = Alignment.CenterVertically) {
            Box(
                modifier = Modifier
                    .size(40.dp)
                    .clip(RoundedCornerRadius(12.dp))
                    .background(Color(0xFF3B82F6).copy(alpha = 0.1f)),
                contentAlignment = Alignment.Center
            ) {
                Icon(
                    imageVector = Icons.Filled.Phone,
                    contentDescription = null,
                    tint = Color(0xFF3B82F6)
                )
            }
            Spacer(modifier = Modifier.width(12.dp))
            Column(modifier = Modifier.weight(1f)) {
                Text(
                    "SIM 卡信息",
                    style = MaterialTheme.typography.titleLarge,
                    fontWeight = FontWeight.Bold
                )
                Text(
                    "点击编辑按钮手动输入本机号码",
                    style = MaterialTheme.typography.bodyMedium,
                    color = MaterialTheme.colorsScheme.onSurfaceVariant
                )
            }
        }
    }

    Spacer(modifier = Modifier.height(16.dp))

    // 提示信息
    Surface(
        modifier = Modifier.fillMaxWidth(),
        color = Color(0xFFFFF3CD).copy(alpha = 0.8f),
        shape = RoundedCornerRadius(12.dp)
    ) {
        Row(
            modifier = Modifier.padding(12.dp),

```

```

        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(
            Icons.Filled.Info,
            contentDescription = null,
            tint = Color(0xFF59E0B)
        )
        Spacer(modifier = Modifier.width(8.dp))
        Text(
            "小米/澎湃OS等定制ROM通常无法自动获取本机号码，请手动输入",
            style = MaterialTheme.typography.bodySmall,
            color = Color(0xFF92400E)
        )
    }
}

Spacer(modifier = Modifier.height(16.dp))

if (simCards.isEmpty()) {
    Text(
        "无法读取 SIM 卡信息",
        style = MaterialTheme.typography.bodyMedium,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )
} else {
    simCards.forEachIndexed { index, simInfo ->
        SimCardItem(
            slot = index + 1,
            phoneNumber = simInfo.phoneNumber,
            carrierName = simInfo.carrierName,
            isCustom = simInfo.isCustom,
            onEdit = {
                showEditDialog = index + 1
                editPhoneNumber = simInfo.phoneNumber ?: ""
            },
            onClear = {
                if (index + 1 == 1) {
                    customSim1Phone = null
                }

                prefs.edit().remove(Constants.PREF_CUSTOM_SIM1_PHONE).apply()
            } else {
                customSim2Phone = null
            }

            prefs.edit().remove(Constants.PREF_CUSTOM_SIM2_PHONE).apply()
        }
    }

    if (index < simCards.size - 1) {
        Spacer(modifier = Modifier.height(12.dp))
    }
}
}
}

```

```

    }

    // 编辑 SIM 号码对话框
    showEditDialog?.let { slot ->
        EditSimPhoneDialog(
            slot = slot,
            currentPhoneNumber = editPhoneNumber,
            onDismiss = { showEditDialog = null },
            onSave = { newPhoneNumber ->
                if (slot == 1) {
                    customSim1Phone = newPhoneNumber.takeIf { it.isNotBlank() }
                    if (newPhoneNumber.isNotBlank()) {
                        prefs.edit().putString(Constants.PREF_CUSTOM_SIM1_PHONE,
newPhoneNumber).apply()
                    } else {
                        prefs.edit().remove(Constants.PREF_CUSTOM_SIM1_PHONE).apply()
                    }
                } else {
                    customSim2Phone = newPhoneNumber.takeIf { it.isNotBlank() }
                    if (newPhoneNumber.isNotBlank()) {
                        prefs.edit().putString(Constants.PREF_CUSTOM_SIM2_PHONE,
newPhoneNumber).apply()
                    } else {
                        prefs.edit().remove(Constants.PREF_CUSTOM_SIM2_PHONE).apply()
                    }
                }
                showEditDialog = null
            }
        )
    }
}

private data class SimCardInfo(
    val phoneNumber: String?,
    val carrierName: String?,
    val isCustom: Boolean = false
)

private fun getSimCardInfo(context: Context, customSim1Phone: String? = null,
customSim2Phone: String? = null): List<SimCardInfo> {
    val simCards = mutableListOf<SimCardInfo>()
    try {
        if (ContextCompat.checkSelfPermission(context, Manifest.permission.READ_PHONE_STATE)
!= PackageManager.PERMISSION_GRANTED) {
            // 即使没有权限，也显示自定义号码
            if (customSim1Phone != null) {
                simCards.add(SimCardInfo(phoneNumber = customSim1Phone, carrierName = null,
isCustom = true))
            }
            if (customSim2Phone != null) {
                simCards.add(SimCardInfo(phoneNumber = customSim2Phone, carrierName = null,
isCustom = true))
            }
        }
    }
}

```

```

        return simCards
    }

    val subscriptionManager = if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.LOLLIPOP_MR1) {
        SubscriptionManager.from(context)
    } else {
        null
    }

    var addedSim1 = false
    var addedSim2 = false

    if (subscriptionManager != null && Build.VERSION.SDK_INT >=
Build.VERSION_CODES.LOLLIPOP_MR1) {
        val activeSubscriptions = subscriptionManager.activeSubscriptionInfoList
        activeSubscriptions?.forEachIndexed { index, subInfo ->
            val slotIndex = index + 1
            val customPhone = if (slotIndex == 1) customSim1Phone else if (slotIndex ==
2) customSim2Phone else null
            val phoneNumber = customPhone ?: subInfo.number?.takeIf { it.isNotBlank() }
            simCards.add(
                SimCardInfo(
                    phoneNumber = phoneNumber,
                    carrierName = subInfo.carrierName?.toString(),
                    isCustom = customPhone != null
                )
            )
            if (slotIndex == 1) addedSim1 = true
            if (slotIndex == 2) addedSim2 = true
        }
    }

    // 如果 SIM1 没有添加但有自定义号码, 添加
    if (!addedSim1 && customSim1Phone != null) {
        simCards.add(SimCardInfo(phoneNumber = customSim1Phone, carrierName = null,
isCustom = true))
        addedSim1 = true
    }

    // 如果 SIM2 没有添加但有自定义号码, 添加
    if (!addedSim2 && customSim2Phone != null) {
        simCards.add(SimCardInfo(phoneNumber = customSim2Phone, carrierName = null,
isCustom = true))
        addedSim2 = true
    }

    // 回退方案: 如果没有获取到任何 SIM 信息但有自定义号码
    if (simCards.isEmpty()) {
        if (customSim1Phone != null) {
            simCards.add(SimCardInfo(phoneNumber = customSim1Phone, carrierName = null,
isCustom = true))
        }
        if (customSim2Phone != null) {

```

```

        simCards.add(SimCardInfo(phoneNumber = customSim2Phone, carrierName = null,
isCustom = true))
    }

    // 如果没有自定义号码, 尝试获取默认号码
    if (simCards.isEmpty()) {
        val telephonyManager = context.getSystemService(Context.TELEPHONY_SERVICE)
as TelephonyManager
        val phoneNumber = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
            telephonyManager.line1Number
        } else {
            @Suppress("DEPRECATION")
            telephonyManager.line1Number
        }
        if (!phoneNumber.isNullOrBlank()) {
            simCards.add(
                SimCardInfo(
                    phoneNumber = phoneNumber,
                    carrierName = telephonyManager.networkOperatorName,
                    isCustom = false
                )
            )
        }
    }
} catch (e: Exception) {
    e.printStackTrace()
    // 即使出错, 也显示自定义号码 (避免重复添加)
    val addedSim1InCatch = simCards.any { it.isCustom }
    if (!addedSim1InCatch && customSim1Phone != null) {
        simCards.add(SimCardInfo(phoneNumber = customSim1Phone, carrierName = null,
isCustom = true))
    }
    val addedSim2InCatch = simCards.size > 1
    if (!addedSim2InCatch && customSim2Phone != null) {
        simCards.add(SimCardInfo(phoneNumber = customSim2Phone, carrierName = null,
isCustom = true))
    }
}
return simCards
}

@Composable
fun SimCardItem(
    slot: Int,
    phoneNumber: String?,
    carrierName: String?,
    isCustom: Boolean = false,
    onEdit: () -> Unit = {},
    onClear: () -> Unit = {}
) {
    Card(
        modifier = Modifier.fillMaxWidth(),

```

```

        shape = RoundedCornersShape(12.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorsScheme.surfaceVariant.copy(alpha = 0.5f)
        )
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Box(
                modifier = Modifier
                    .size(40.dp)
                    .clip(CircleShape)
                    .background(if (isCustom) Color(0xFF10B981).copy(alpha = 0.2f) else
Color(0xFF3B82F6).copy(alpha = 0.2f)),
                contentAlignment = Alignment.Center
            ) {
                Text(
                    "SIM$slot",
                    style = MaterialTheme.typography.bodyMedium,
                    fontWeight = FontWeight.Bold,
                    color = if (isCustom) Color(0xFF10B981) else Color(0xFF3B82F6)
                )
            }
            Spacer(modifier = Modifier.width(12.dp))
            Column(modifier = Modifier.weight(1f)) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Text(
                        phoneNumber ?: "无法获取号码",
                        style = MaterialTheme.typography.bodyLarge,
                        fontWeight = FontWeight.Medium,
                        color = if (phoneNumber != null) {
                            MaterialTheme.colorsScheme.onSurface
                        } else {
                            MaterialTheme.colorsScheme.onSurfaceVariant
                        }
                    )
                    if (isCustom) {
                        Spacer(modifier = Modifier.width(8.dp))
                        Surface(
                            color = Color(0xFF10B981).copy(alpha = 0.2f),
                            shape = RoundedCornersShape(4.dp)
                        ) {
                            Text(
                                "自定义",
                                style = MaterialTheme.typography.labelSmall,
                                color = Color(0xFF10B981),
                                modifier = Modifier.padding(horizontal = 6.dp, vertical =
2.dp)
                            )
                        }
                    }
                }
            }
        }
    }
}

```



```

        slot: Int,
        currentPhoneNumber: String,
        onDismiss: () -> Unit,
        onSave: (String) -> Unit
    ) {
        var phoneNumber by remember { mutableStateOf(currentPhoneNumber) }

        ModernAlertDialog(
            onDismissRequest = onDismiss,
            title = "设置 SIM$slot 号码",
            content = {
                Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {
                    Text(
                        "如果无法自动获取本机号码，可以手动输入",
                        style = MaterialTheme.typography.bodyMedium,
                        color = MaterialTheme.colorsScheme.onSurfaceVariant
                    )
                    OutlinedTextField(
                        value = phoneNumber,
                        onValueChange = { phoneNumber = it },
                        label = { Text("本机号码") },
                        placeholder = { Text("例如: 13800138000") },
                        modifier = Modifier.fillMaxWidth(),
                        leadingIcon = {
                            Icon(Icons.Outlined.Phone, contentDescription = null)
                        },
                        shape = RoundedCornerShape(12.dp),
                        singleLine = true
                    )
                }
            },
            confirmButton = {
                Button(
                    onClick = { onSave(phoneNumber) },
                    shape = RoundedCornerShape(12.dp)
                ) { Text("保存") }
            },
            dismissButton = {
                TextButton(onClick = onDismiss) { Text("取消") }
            }
        )
    }
}

```

```

// =====
// 新的标签页组件
// =====

```

```

@Composable
fun HomeTab(
    isEnabled: Boolean,
    onEnabledChange: (Boolean) -> Unit,
    startOnBoot: Boolean,
    onStartOnBootChange: (Boolean) -> Unit,

```

```

smsGranted: Boolean,
notifGranted: Boolean,
isIgnoringBatteryOptimizations: Boolean,
onRequestSmsPermission: () -> Unit,
onRequestNotificationPermission: () -> Unit,
onRequestBatteryOptimization: () -> Unit
) {
    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding(horizontal = 16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp),
        contentPadding = PaddingValues(vertical = 16.dp)
    ) {
        // Service Status Card
        item {
            ModernCard(
                modifier = Modifier.fillMaxWidth()
            ) {
                Column(modifier = Modifier.padding(20.dp)) {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        Column(modifier = Modifier.weight(1f)) {
                            Text(
                                "转发服务",
                                style = MaterialTheme.typography.titleLarge,
                                fontWeight = FontWeight.Bold
                            )
                            Spacer(Modifier.height(6.dp))
                            Row(verticalAlignment = Alignment.CenterVertically) {
                                val statusColor = if (isEnabled) Color(0xFF10B981) else
                                color(0xFF9CA3AF)
                                Box(
                                    modifier = Modifier
                                        .size(12.dp)
                                        .background(statusColor, shape = CircleShape)
                                )
                                Spacer(Modifier.width(8.dp))
                                Text(
                                    if (isEnabled) "服务运行中" else "服务已停止",
                                    style = MaterialTheme.typography.bodyMedium,
                                    color = MaterialTheme.colorsScheme.onSurfaceVariant
                                )
                            }
                        }
                    }
                    val switchScale by animateFloatAsState(
                        targetValue = if (isEnabled) 1.1f else 1f,
                        animationSpec = spring(stiffness = Spring.StiffnessMedium),
                        label = "switchAnimation"
                    )
                    Switch(

```

```

        checked = isEnabled,
        onCheckedChange = onEnabledChange,
        modifier = Modifier.scale(switchScale),
        colors = SwitchDefaults.colors(
            checkedThumbColor = Color.White,
            checkedTrackColor = Color(0xFF667EEA)
        )
    )
}

Spacer(modifier = Modifier.height(20.dp))

// Permission items
PermissionItem(
    icon = Icons.Outlined.Notifications,
    title = "通知权限",
    granted = notifGranted,
    onClick = onRequestNotificationPermission
)
Spacer(modifier = Modifier.height(12.dp))
PermissionItem(
    icon = Icons.Outlined.Sms,
    title = "短信权限",
    granted = smsGranted,
    onClick = onRequestSmsPermission
)
Spacer(modifier = Modifier.height(12.dp))
PermissionItem(
    icon = Icons.Outlined.BatteryFull,
    title = "电池优化白名单",
    granted = isIgnoringBatteryOptimizations,
    onClick = onRequestBatteryOptimization
)

Spacer(modifier = Modifier.height(20.dp))

HorizontalDivider()

Spacer(modifier = Modifier.height(16.dp))

Row(
    verticalAlignment = Alignment.CenterVertically,
    modifier = Modifier.fillMaxWidth()
) {
    Icon(
        Icons.Outlined.PowerSettingsNew,
        contentDescription = null,
        tint = MaterialTheme.colorScheme.onSurfaceVariant
    )
    Spacer(modifier = Modifier.width(12.dp))
    Column(modifier = Modifier.weight(1f)) {
        Text(
            "开机自启动",

```



```

        contentAlignment = Alignment.Center
    ) {
        Icon(
            imageVector = Icons.Filled.Rule,
            contentDescription = null,
            tint = Color(0xFF10B981)
        )
    }
    Spacer(modifier = Modifier.width(12.dp))
    Column {
        Text(
            "关键词配置",
            style = MaterialTheme.typography.titleLarge,
            fontWeight = FontWeight.Bold
        )
        Text(
            "设置转发关键词",
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorsScheme.onSurfaceVariant
        )
    }
}

outlinedTextField(
    value = newKeywordInput,
    onValueChange = onNewKeywordInputChange,
    label = { Text("输入关键词（留空表示全部）") },
    modifier = Modifier.fillMaxWidth(),
    leadingIcon = {
        Icon(Icons.Outlined.Search, contentDescription = null)
    },
    shape = RoundedCornerShape(12.dp)
)

ExposedDropdownMenuBox(
    expanded = configChannelDropdownExpanded,
    onExpandedChange = onConfigChannelDropdownExpandedChange
) {
    outlinedTextField(
        value = channels.find { it.id == selectedChannelIdForNewCfg
}?.name ?: "选择转发通道",
        onValueChange = {},
        readOnly = true,
        label = { Text("转发通道") },
        modifier = Modifier.menuAnchor().fillMaxWidth(),
        leadingIcon = {
            Icon(Icons.Outlined.Send, contentDescription = null)
        },
        trailingIcon = {
ExposedDropdownMenuDefaults.TrailingIcon(configChannelDropdownExpanded) },
        shape = RoundedCornerShape(12.dp)
    )
    ExposedDropdownMenu(

```

```

        expanded = configChannelDropdownExpanded,
        onRequestDismissRequest = {
            onConfigChannelDropdownExpandedChange(false) }
    ) {
        channels.forEach { ch ->
            DropdownMenuItem(
                text = { Text(ch.name) },
                onClick = {
                    onSelectChannelIdForNewCfgChange(ch.id)
                    onConfigChannelDropdownExpandedChange(false)
                }
            )
        }
        if (channels.isEmpty()) {
            DropdownMenuItem(
                text = { Text("请先添加通道") },
                onClick = { onConfigChannelDropdownExpandedChange(false) }
            )
        }
    }
}

Button(
    onClick = onAddConfig,
    modifier = Modifier.fillMaxWidth(),
    shape = RoundedCornerShape(12.dp),
    contentPadding = PaddingValues(vertical = 14.dp)
) {
    Icon(Icons.Outlined.Add, contentDescription = null)
    Spacer(modifier = Modifier.width(8.dp))
    Text("添加配置", fontSize = 16.sp)
}

// Config list
if (configs.isNotEmpty()) {
    Spacer(modifier = Modifier.height(8.dp))
    configs.forEach { cfg ->
        val chName = channels.find { it.id == cfg.channelId }?.name ?: "
(已删除通道)"

        ConfigItem(
            keyword = cfg.keyword,
            channelName = chName,
            onEdit = { onEditConfig(cfg) },
            onDelete = { onDeleteConfig(cfg) }
        )
    }
}
}
}
}
}
}
}

```

```

@Composable
fun ChannelTab(
    channels: List<Channel>,
    newChannelName: String,
    onNewChannelNameChange: (String) -> Unit,
    newChannelTarget: String,
    onNewChannelTargetChange: (String) -> Unit,
    newChannelType: ChannelType,
    onNewChannelTypeChange: (ChannelType) -> Unit,
    channelTypeExpanded: Boolean,
    onChannelTypeExpandedChange: (Boolean) -> Unit,
    onAddChannel: () -> Unit,
    onDeleteChannel: (Channel) -> Unit,
    onEditChannel: (Channel) -> Unit
) {
    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding(horizontal = 16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp),
        contentPadding = PaddingValues(vertical = 16.dp)
    ) {
        item {
            ModernCard {
                Column(
                    modifier = Modifier.padding(20.dp),
                    verticalArrangement = Arrangement.spacedBy(16.dp)
                ) {
                    Row(verticalAlignment = Alignment.CenterVertically) {
                        Box(
                            modifier = Modifier
                                .size(40.dp)
                                .clip(RoundedCornerRadius(12.dp))
                                .background(Color(0xFF667EEA).copy(alpha = 0.1f)),
                            contentAlignment = Alignment.Center
                        ) {
                            Icon(
                                imageVector = Icons.Filled.Cloud,
                                contentDescription = null,
                                tint = Color(0xFF667EEA)
                            )
                        }
                        Spacer(modifier = Modifier.width(12.dp))
                        Column {
                            Text(
                                "转发通道管理",
                                style = MaterialTheme.typography.titleLarge,
                                fontWeight = FontWeight.Bold
                            )
                            Text(
                                "添加和管理转发通道",
                                style = MaterialTheme.typography.bodyMedium,

```



```

        color = MaterialTheme.colorsScheme.onSurfaceVariant
    )
}

outlinedTextField(
    value = newChannelName,
    onValueChange = onNewChannelNameChange,
    label = { Text("通道名称") },
    modifier = Modifier.fillMaxWidth(),
    leadingIcon = {
        Icon(Icons.Outlined.Label, contentDescription = null)
    },
    shape = RoundedCornerShape(12.dp)
)

ExposedDropdownMenuBox(
    expanded = channelTypeExpanded,
    onExpandedChange = onChannelTypeExpandedChange
) {
    outlinedTextField(
        value = getChannelTypeLabel(newChannelType),
        onValueChange = {},
        readOnly = true,
        label = { Text("通道类型") },
        modifier = Modifier.menuAnchor().fillMaxWidth(),
        leadingIcon = {
            Icon(Icons.Outlined.Category, contentDescription = null)
        },
        trailingIcon = {
ExposedDropdownMenuDefaults.TrailingIcon(channelTypeExpanded) },
        shape = RoundedCornerShape(12.dp)
    )
    ExposedDropdownMenu(
        expanded = channelTypeExpanded,
        onDismissRequest = { onChannelTypeExpandedChange(false) }
    ) {
        ChannelType.entries.forEach { t ->
            DropdownMenuItem(
                text = { Text(getChannelTypeLabel(t)) },
                onClick = {
                    onNewChannelTypeChange(t)
                    onChannelTypeExpandedChange(false)
                }
            )
        }
    }
}

outlinedTextField(
    value = newChannelTarget,
    onValueChange = onNewChannelTargetChange,
    label = { Text("webhook 地址") },

```



```

) {
    // SIM 卡信息卡片
    item {
        SimCardInfoCard()
    }

    // 消息格式配置
    item {
        ModernCard(
            modifier = Modifier.fillMaxWidth()
        ) {
            Column(modifier = Modifier.padding(20.dp)) {
                Text(
                    "消息格式",
                    style = MaterialTheme.typography.titleLarge,
                    fontWeight = FontWeight.Bold
                )

                Spacer(modifier = Modifier.height(16.dp))

                // 显示本机号码
                Row(
                    verticalAlignment = Alignment.CenterVertically,
                    modifier = Modifier.fillMaxWidth()
                ) {
                    Icon(
                        Icons.Outlined.Phone,
                        contentDescription = null,
                        tint = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                    Spacer(modifier = Modifier.width(12.dp))
                    Column(modifier = Modifier.weight(1f)) {
                        Text(
                            "显示本机号码",
                            style = MaterialTheme.typography.bodyLarge,
                            fontWeight = FontWeight.Medium
                        )
                        Text(
                            "转发时显示接收短信的本机号码",
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }
                Switch(
                    checked = showReceiverPhone,
                    onCheckedChange = onShowReceiverPhoneChange
                )
            }

            Spacer(modifier = Modifier.height(12.dp))

            // 显示发送者号码
            Row(

```

```

        verticalAlignment = Alignment.CenterVertically,
        modifier = Modifier.fillMaxWidth()
    ) {
        Icon(
            Icons.Outlined.Person,
            contentDescription = null,
            tint = MaterialTheme.colorScheme.onSurfaceVariant
        )
        Spacer(modifier = Modifier.width(12.dp))
        Column(modifier = Modifier.weight(1f)) {
            Text(
                "显示发送者号码",
                style = MaterialTheme.typography.bodyLarge,
                fontWeight = FontWeight.Medium
            )
            Text(
                "转发时显示短信发送者号码",
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
        }
        Switch(
            checked = showSenderPhone,
            onCheckedChange = onShowSenderPhoneChange
        )
    }

    Spacer(modifier = Modifier.height(12.dp))

    // 突出显示验证码
    Row(
        verticalAlignment = Alignment.CenterVertically,
        modifier = Modifier.fillMaxWidth()
    ) {
        Icon(
            Icons.Outlined.VpnKey,
            contentDescription = null,
            tint = MaterialTheme.colorScheme.onSurfaceVariant
        )
        Spacer(modifier = Modifier.width(12.dp))
        Column(modifier = Modifier.weight(1f)) {
            Text(
                "突出显示验证码",
                style = MaterialTheme.typography.bodyLarge,
                fontWeight = FontWeight.Medium
            )
            Text(
                "自动识别并突出显示短信验证码",
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
        }
    }
    Switch(

```

```

        checked = highlightVerificationCode,
        onCheckedChange = onHighlightVerificationCodeChange
    )
    }
    }
}

// 工具
item {
    ModernCard(
        modifier = Modifier.fillMaxWidth()
    ) {
        Column(modifier = Modifier.padding(20.dp)) {
            Text(
                "工具",
                style = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.Bold
            )

            Spacer(modifier = Modifier.height(16.dp))

            Button(
                onClick = onShowTestDialog,
                modifier = Modifier.fillMaxWidth(),
                shape = RoundedCornerShape(12.dp),
                contentPadding = PaddingValues(vertical = 14.dp)
            ) {
                Icon(Icons.Outlined.Science, contentDescription = null)
                Spacer(modifier = Modifier.width(8.dp))
                Text("测试规则", fontSize = 16.sp)
            }
        }
    }
}
}

fun getChannelTypeLabel(type: ChannelType): String = when (type) {
    ChannelType.WECHAT -> "企业微信"
    ChannelType.DINGTALK -> "钉钉"
    ChannelType.FEISHU -> "飞书"
    ChannelType.GENERIC_WEBHOOK -> "webhook"
}

private fun loadChannels(prefs: android.content.SharedPreferences): List<Channel> {
    val arrStr = prefs.getString(Constants.PREF_CHANNELS, "[]") ?: "[]"
    return try {
        val arr = JSONArray(arrStr)
        (0 until arr.length()).map { i ->
            val o = arr.getJSONObject(i)
            val typeStr = o.optString("type", "WECHAT")

```

```

        val type = try { ChannelType.valueOf(typeStr) } catch (t: Throwable) {
ChannelType.WECHAT }
        Channel(o.getString("id"), o.getString("name"), type, o.getString("target"))
    }
} catch (t: Throwable) {
    emptyList()
}
}

private fun saveChannels(prefs: android.content.SharedPreferences, channels: List<Channel>)
{
    val arr = JSONArray()
    channels.forEach {
        val o = JSONObject()
        o.put("id", it.id)
        o.put("name", it.name)
        o.put("type", it.type.name)
        o.put("target", it.target)
        arr.put(o)
    }
    prefs.edit().putString(Constants.PREF_CHANNELS, arr.toString()).apply()
}

private fun loadConfigs(prefs: android.content.SharedPreferences): List<KeywordConfig> {
    val arrStr = prefs.getString(Constants.PREF_KEYWORD_CONFIGS, "[]") ?: "[]"
    return try {
        val arr = JSONArray(arrStr)
        (0 until arr.length()).map { i ->
            val o = arr.getJSONObject(i)
            KeywordConfig(o.getString("id"), o.getString("keyword"),
o.getString("channelId"))
        }
    } catch (t: Throwable) {
        emptyList()
    }
}

private fun saveConfigs(prefs: android.content.SharedPreferences, configs:
List<KeywordConfig>) {
    val arr = JSONArray()
    configs.forEach {
        val o = JSONObject()
        o.put("id", it.id)
        o.put("keyword", it.keyword)
        o.put("channelId", it.channelId)
        arr.put(o)
    }
    prefs.edit().putString(Constants.PREF_KEYWORD_CONFIGS, arr.toString()).apply()
}

```

models.kt

```

/*
 * 短信转发助手
 * 版本: V2.7.2
 *
 * 著作权人: 华昊科技有限公司
 * 开发者: 王士辉
 *
 * Copyright (c) 2026 华昊科技有限公司. All rights reserved.
 * 联系邮箱: huahao@email.cn
 */

package com.lanbing.smsforwarder

/**
 * Shared model types for channels and keyword rules.
 */
enum class ChannelType { WECHAT, DINGTALK, FEISHU, GENERIC_WEBHOOK }

data class Channel(
    val id: String,
    val name: String,
    val type: ChannelType,
    val target: String // webhook URL
)

data class KeywordConfig(
    val id: String,
    val keyword: String, // empty string means match-all
    val channelId: String
)

```

NetworkChangeReceiver.kt

```

/*
 * 短信转发助手
 * 版本: V2.7.2
 *
 * 著作权人: 华昊科技有限公司
 * 开发者: 王士辉
 *
 * Copyright (c) 2026 华昊科技有限公司. All rights reserved.
 * 联系邮箱: huahao@email.cn
 */

package com.lanbing.smsforwarder

import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.net.ConnectivityManager
import android.net.NetworkCapabilities

```

```

import android.os.Build
import android.util.Log

/**
 * NetworkChangeReceiver: 监听网络状态变化，网络恢复时触发失败消息重试
 */
class NetworkChangeReceiver : BroadcastReceiver() {

    companion object {
        private const val TAG = "NetworkChangeReceiver"
        private var lastRetryTime = 0L
        private var lastNetworkState = false
    }

    override fun onReceive(context: Context, intent: Intent) {
        if (intent.action != ConnectivityManager.CONNECTIVITY_ACTION) return

        val isAvailable = isNetworkAvailable(context)
        val now = System.currentTimeMillis()

        // 防抖处理：只在网络从不可用变为可用，且距离上次重试超过 NETWORK_DEBOUNCE_MS 时才重试
        if (isAvailable && !lastNetworkState && (now - lastRetryTime >
Constants.NETWORK_DEBOUNCE_MS)) {
            Log.d(TAG, "网络已恢复，触发失败消息重试")
            lastRetryTime = now
            SmsReceiver.retryFailedMessages(context)
        }

        lastNetworkState = isAvailable
    }

    private fun isNetworkAvailable(context: Context): Boolean {
        return try {
            val connectivityManager = context.getSystemService(Context.CONNECTIVITY_SERVICE)
as ConnectivityManager
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
                val network = connectivityManager.activeNetwork ?: return false
                val capabilities = connectivityManager.getNetworkCapabilities(network) ?:
return false
                capabilities.hasCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET) &&
capabilities.hasCapability(NetworkCapabilities.NET_CAPABILITY_VALIDATED)
            } else {
                @Suppress("DEPRECATION")
                val networkInfo = connectivityManager.activeNetworkInfo
                networkInfo?.isConnected == true
            }
        } catch (t: Throwable) {
            Log.e(TAG, "Error checking network availability", t)
            false
        }
    }
}

```


SmsForegroundService.kt

```
/*
 * 短信转发助手
 * 版本: v2.7.2
 *
 * 著作权人: 华昊科技有限公司
 * 开发者: 王士辉
 *
 * Copyright (c) 2026 华昊科技有限公司. All rights reserved.
 * 联系邮箱: huahao@email.cn
 */

package com.lanbing.smsforwarder

import android.app.Notification
import import android.app.NotificationChannel
import import android.app.NotificationManager
import import android.app.Service
import import android.content.BroadcastReceiver
import import android.content.Context
import import android.content.Intent
import import android.content.IntentFilter
import import android.graphics.BitmapFactory
import import android.net.Uri
import import android.os.Build
import import android.os.IBinder
import import androidx.core.app.NotificationCompat
import import androidx.core.app.NotificationManagerCompat
import import android.app.PendingIntent
import import android.provider.Settings
import import android.util.Log

class SmsForegroundService : Service() {

    companion object {
        const val ACTION_UPDATE = "com.lanbing.smsforwarder.ACTION_LOG_UPDATED"
        const val ACTION_STOP = "com.lanbing.smsforwarder.ACTION_STOP_SERVICE"
        private const val TAG = "SmsForegroundService"
        private var lastNotificationUpdateTime = 0L
    }

    private val updateReceiver = object : BroadcastReceiver() {
        override fun onReceive(context: Context?, intent: Intent?) {
            try {
                val action = intent?.action
                if (action == ACTION_STOP) {
                    stopSelf()
                    LogStore.append(applicationContext, "收到通知停止服务请求, 服务已停止")
                    return
                }
            }
        }
    }
}
```

```

        }
        updateNotification()
    } catch (t: Throwable) {
        Log.w(TAG, "updateNotification failed", t)
    }
}

}

override fun onCreate() {
    super.onCreate()
    createChannel()
    try {
        val filter = IntentFilter().apply {
            addAction(ACTION_UPDATE)
            addAction(ACTION_STOP)
        }
        registerReceiver(updateReceiver, filter)
    } catch (t: Throwable) {
        Log.w(TAG, "registerReceiver failed", t)
    }
}

private fun createChannel() {
    try {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val nm = getSystemService(NotificationManager::class.java)
            if (nm != null) {
                val importance = NotificationManager.IMPORTANCE_HIGH
                val channel = NotificationChannel(
                    Constants.NOTIFICATION_CHANNEL_ID,
                    Constants.NOTIFICATION_CHANNEL_NAME,
                    importance
                )
                channel.setShowBadge(false)
                channel.lockscreenVisibility =
android.app.Notification.VISIBILITY_PRIVATE
                nm.createNotificationChannel(channel)
            } else {
                Log.w(TAG, "NotificationManager is null when creating channel")
            }
        }
    } catch (t: Throwable) {
        Log.w(TAG, "createChannel failed", t)
    }
}

override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
    // 检查通知权限
    if (!NotificationManagerCompat.from(this).areNotificationsEnabled()) {
        Log.w(TAG, "Notification permission not granted, cannot start foreground
service")
        LogStore.append(applicationContext, "错误: 缺少通知权限, 无法启动前台服务")
        stopSelf()
    }
}

```

```

        return START_NOT_STICKY
    }

    val notification: Notification = try {
        buildNotification()
    } catch (t: Throwable) {
        Log.w(TAG, "buildNotification failed, use fallback", t)
        // fallback: 直接使用编译时资源, 确保 smallIcon 不会回退到系统占位
        NotificationCompat.Builder(this, Constants.NOTIFICATION_CHANNEL_ID)
            .setContentTitle("短信转发助手")
            .setContentText("服务正在运行")
            .setSmallIcon(R.drawable.ic_stat_notification)
            .setOngoing(true)
            .build()
    }

    try {
        if (Build.VERSION.SDK_INT >= 34) {
            val type = getRemoteMessagingForegroundServiceType()
            if (type != 0) {
                startForeground(Constants.NOTIFICATION_ID, notification, type)
            } else {
                Log.w(TAG, "FOREGROUND_SERVICE_TYPE_REMOTE_MESSAGING not found via
reflection, calling startForeground without type")
                startForeground(Constants.NOTIFICATION_ID, notification)
            }
        } else {
            startForeground(Constants.NOTIFICATION_ID, notification)
        }
    } catch (t: Throwable) {
        Log.w(TAG, "startForeground failed, stopping service", t)
        LogStore.append(applicationContext, "ERROR: startForeground failed:
${t.javaClass.simpleName} ${t.message}")
        stopSelf()
        return START_NOT_STICKY
    }

    try {
        val nm = getSystemService(NotificationManager::class.java)
        val channel = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O)
nm?.getNotificationChannel(Constants.NOTIFICATION_CHANNEL_ID) else null
        val chInfo = if (channel != null) "channel(${channel.id}):
importance=${channel.importance} name=${channel.name}" else "channel:null"
        val notifAllowed =
NotificationManagerCompat.from(this).areNotificationsEnabled()
        LogStore.append(applicationContext, "DEBUG: notifAllowed=$notifAllowed ;
$chInfo")
    } catch (t: Throwable) {
        LogStore.append(applicationContext, "DEBUG: 检查 channel 失败: ${t.message}")
    }

    try {
        val nm2 = getSystemService(Context.NOTIFICATION_SERVICE) as? NotificationManager

```

```

        nm2?.notify(Constants.NOTIFICATION_ID, notification)
    } catch (t: Throwable) {
        Log.w(TAG, "extra notify failed", t)
    }
    return START_STICKY
}

private fun getRemoteMessagingForegroundServiceType(): Int {
    return try {
        val cls = Class.forName("android.app.ServiceInfo")
        val field = cls.getField("FOREGROUND_SERVICE_TYPE_REMOTE_MESSAGING")
        (field.getInt(null))
    } catch (t: Throwable) {
        Log.w(TAG, "Failed to read FOREGROUND_SERVICE_TYPE_REMOTE_MESSAGING via
reflection: ${t.message}")
        0
    }
}

/**
 * 保留 resolveSmallIcon 作为极小概率回退（仍优先使用编译时资源），但通知构建处已直接使用
R.drawable.ic_stat_notification
 */
private fun resolveSmallIcon(): Int {
    val statDrawable = resources.getIdentifier("ic_stat_notification", "drawable",
packageName)
    if (statDrawable != 0) return statDrawable
    val statMipmap = resources.getIdentifier("ic_stat_notification", "mipmap",
packageName)
    if (statMipmap != 0) return statMipmap

    val appIcon = applicationInfo.icon
    if (appIcon != 0) return appIcon
    return android.R.drawable.ic_dialog_info
}

private fun buildNotification(): Notification {
    val prefs = getSharedPreferences(Constants.PREFS_NAME, Context.MODE_PRIVATE)
    val enabled = prefs.getBoolean(Constants.PREF_ENABLED, false)
    val status = if (enabled) "已启用" else "已禁用"
    val latest = LogStore.latest(this)

    val builder = NotificationCompat.Builder(this, Constants.NOTIFICATION_CHANNEL_ID)
        .setContentTitle("短信转发助手 - $status")
        .setContentText(latest)
        .setSmallIcon(R.drawable.ic_stat_notification)
        .setOngoing(true)
        .setCategory(NotificationCompat.CATEGORY_SERVICE)
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setOnlyAlertOnce(true)

    // 尝试设置彩色大图标（在展开的通知/设置中会显示），优先使用 mipmap/ic_launcher 或
drawable/ic_launcher

```

```

        try {
            val largeId = resources.getIdentifier("ic_launcher", "mipmap",
packageName).takeIf { it != 0 }
                ?: resources.getIdentifier("ic_launcher", "drawable", packageName).takeIf {
it != 0 }
            if (largeId != null && largeId != 0) {
                val bmp = BitmapFactory.decodeResource(resources, largeId)
                if (bmp != null) builder.setLargeIcon(bmp)
            }
        } catch (t: Throwable) {
            Log.w(TAG, "setLargeIcon failed: ${t.message}")
        }

        val piFlags = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
            PendingIntent.FLAG_IMMUTABLE or PendingIntent.FLAG_UPDATE_CURRENT
        } else {
            PendingIntent.FLAG_UPDATE_CURRENT
        }

        val mainIntent = Intent(this, MainActivity::class.java).apply {
            flags = Intent.FLAG_ACTIVITY_CLEAR_TOP or Intent.FLAG_ACTIVITY_SINGLE_TOP
            `package` = packageName
        }
        val pendingIntent = PendingIntent.getActivity(this, 0, mainIntent, piFlags)
        builder.setContentIntent(pendingIntent)

        val stopIntent = Intent(ACTION_STOP).apply { `package` = packageName }
        val stopPending = PendingIntent.getBroadcast(this, 1, stopIntent, piFlags)
        builder.addAction(android.R.drawable.ic_menu_close_clear_cancel, "停止服务",
stopPending)

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val intent = Intent(Settings.ACTION_CHANNEL_NOTIFICATION_SETTINGS).apply {
                putExtra(Settings.EXTRA_APP_PACKAGE, packageName)
                putExtra(Settings.EXTRA_CHANNEL_ID, Constants.NOTIFICATION_CHANNEL_ID)
                flags = Intent.FLAG_ACTIVITY_NEW_TASK
                `package` = packageName
            }
            val pi = PendingIntent.getActivity(this, 2, intent, piFlags)
            builder.addAction(android.R.drawable.ic_menu_manage, "通知设置", pi)
        } else {
            val intent = Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS).apply {
                data = Uri.parse("package:$packageName")
                flags = Intent.FLAG_ACTIVITY_NEW_TASK
                `package` = packageName
            }
            val pi = PendingIntent.getActivity(this, 3, intent, piFlags)
            builder.addAction(android.R.drawable.ic_menu_manage, "应用设置", pi)
        }

        return builder.build()
    }

```

```

private fun updateNotification() {
    val now = System.currentTimeMillis()
    if (now - lastNotificationUpdateTime < Constants.NOTIFICATION_UPDATE_THROTTLE_MS) {
        Log.d(TAG, "Skipping notification update due to throttling")
        return
    }
    lastNotificationUpdateTime = now
    try {
        val nm = getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
        nm.notify(Constants.NOTIFICATION_ID, buildNotification())
    } catch (t: Throwable) {
        Log.w(TAG, "updateNotification failed", t)
    }
}

override fun onDestroy() {
    super.onDestroy()
    try { unregisterReceiver(updateReceiver) } catch (e: Exception) { /* ignore */ }
}

override fun onBind(intent: Intent?): IBinder? = null
}

```

SmsReceiver.kt

```

/*
 * 短信转发助手
 * 版本: v2.7.2
 *
 * 著作权人: 华昊科技有限公司
 * 开发者: 王士辉
 *
 * Copyright (c) 2026 华昊科技有限公司. All rights reserved.
 * 联系邮箱: huahao@email.cn
 */

package com.lanbing.smsforwarder

import android.Manifest
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.content.pm.PackageManager
import android.os.Build
import android.provider.Telephony
import android.telephony.SubscriptionManager
import android.telephony.TelephonyManager
import android.util.Log
import androidx.core.content.ContextCompat
import okhttp3.MediaType.Companion.toMediaType
import okhttp3.OkHttpClient

```

```

import okhttp3.Request
import okhttp3.RequestBody.Companion.toRequestBody
import org.json.JSONArray
import org.json.JSONObject
import java.io.File
import java.net.MalformedURLException
import java.net.URL
import java.util.concurrent.ConcurrentHashMap
import java.util.concurrent.Executors
import java.util.concurrent.TimeUnit

/**
 * SmsReceiver:
 * - 读取 SharedPreferences 中的 channels / keyword_configs
 * - 对所有规则逐条匹配（空 keyword 表示匹配全部）
 * - 对每条匹配项并行发送（允许同一条短信被多次发送到相同/不同通道）
 * - 支持 webhook 类型：企业微信、钉钉、飞书、通用 webhook
 * - 添加消息去重机制和失败重试队列
 * - 失败消息持久化到文件
 */

class SmsReceiver : BroadcastReceiver() {

    companion object {
        private const val TAG = "SmsReceiver"

        val client: OkHttpClient = OkHttpClient.Builder()
            .callTimeout(Constants.CALL_TIMEOUT_SECONDS, TimeUnit.SECONDS)
            .connectTimeout(Constants.CONNECT_TIMEOUT_SECONDS, TimeUnit.SECONDS)
            .readTimeout(Constants.READ_TIMEOUT_SECONDS, TimeUnit.SECONDS)
            .build()

        // 固定线程池避免线程爆炸
        private val executor = Executors.newFixedThreadPool(Constants.THREAD_POOL_SIZE)

        // 消息去重缓存: key = sender+content_hash, value = timestamp
        private val recentMessages = ConcurrentHashMap<String, Long>()
        private var lastCleanupTime = 0L
        private const val CLEANUP_INTERVAL_MS = 60000L // 1分钟清理一次

        // 失败消息队列，等待网络恢复时重试
        private val failedMessages = mutableListOf<FailedMessage>()
        private val failedMessageLock = Object()

        data class FailedMessage(
            val channelId: String,
            val channelName: String,
            val channelType: String,
            val channelTarget: String,
            val sender: String,
            val content: String,
            val receiverPhoneNumber: String?,
            val showSenderPhone: Boolean,

```

```

        val highlightVerificationCode: Boolean,
        val timestamp: Long,
        val retryCount: Int = 0
    ) {
        fun toJsonObject(): JsonObject {
            val obj = JsonObject()
            obj.put("channelId", channelId)
            obj.put("channelName", channelName)
            obj.put("channelType", channelType)
            obj.put("channelTarget", channelTarget)
            obj.put("sender", sender)
            obj.put("content", content)
            obj.put("receiverPhoneNumber", receiverPhoneNumber)
            obj.put("showSenderPhone", showSenderPhone)
            obj.put("highlightVerificationCode", highlightVerificationCode)
            obj.put("timestamp", timestamp)
            obj.put("retryCount", retryCount)
            return obj
        }
    }

    companion object {
        fun fromJsonObject(obj: JsonObject): FailedMessage {
            return FailedMessage(
                channelId = obj.getString("channelId"),
                channelName = obj.getString("channelName"),
                channelType = obj.getString("channelType"),
                channelTarget = obj.getString("channelTarget"),
                sender = obj.getString("sender"),
                content = obj.getString("content"),
                receiverPhoneNumber = obj.optString("receiverPhoneNumber", null),
                showSenderPhone = obj.optBoolean("showSenderPhone", true),
                highlightVerificationCode =
obj.optBoolean("highlightVerificationCode", true),
                timestamp = obj.getLong("timestamp"),
                retryCount = obj.getInt("retryCount")
            )
        }

        fun fromChannel(channel: Channel, sender: String, content: String,
receiverPhoneNumber: String?, showSenderPhone: Boolean, highlightVerificationCode: Boolean,
timestamp: Long, retryCount: Int = 0): FailedMessage {
            return FailedMessage(
                channelId = channel.id,
                channelName = channel.name,
                channelType = channel.type.name,
                channelTarget = channel.target,
                sender = sender,
                content = content,
                receiverPhoneNumber = receiverPhoneNumber,
                showSenderPhone = showSenderPhone,
                highlightVerificationCode = highlightVerificationCode,
                timestamp = timestamp,
                retryCount = retryCount
            )
        }
    }

```



```

        )
    }
}

fun toChannel(): Channel {
    val type = try { ChannelType.valueOf(channelType) } catch (t: Throwable) {
ChannelType.WECHAT }
    return Channel(channelId, channelName, type, channelTarget)
}

private fun failedMessagesFile(context: Context): File {
    return File(context.filesDir, Constants.FAILED_MESSAGES_FILE)
}

private fun saveFailedMessages(context: Context) {
    synchronized(failedMessageLock) {
        try {
            val file = failedMessagesFile(context)
            val arr = JSONArray()
            failedMessages.take(Constants.MAX_FAILED_MESSAGES).forEach {
arr.put(it.toJSONString()) }
            file.writeText(arr.toString())
        } catch (t: Throwable) {
            Log.e(TAG, "Failed to save failed messages", t)
        }
    }
}

private fun loadFailedMessages(context: Context) {
    synchronized(failedMessageLock) {
        try {
            val file = failedMessagesFile(context)
            if (!file.exists()) return
            val arr = JSONArray(file.readText())
            failedMessages.clear()
            for (i in 0 until arr.length()) {

failedMessages.add(FailedMessage.fromJSONObject(arr.getJSONObject(i)))
            }
        } catch (t: Throwable) {
            Log.e(TAG, "Failed to load failed messages", t)
        }
    }
}

private fun cleanupRecentMessages() {
    val now = System.currentTimeMillis()
    if (now - lastCleanupTime < CLEANUP_INTERVAL_MS) return
    lastCleanupTime = now
    recentMessages.entries.removeIf { (now - it.value) >
Constants.DUPLICATE_WINDOW_MS * 2 }
}

```

```

// 供 NetworkChangeReceiver 调用，重试失败的消息
@JvmStatic
fun retryFailedMessages(context: Context) {
    loadFailedMessages(context)
    synchronized(failedMessageLock) {
        if (failedMessages.isEmpty()) return

        val toRetry = failedMessages.filter { it.retryCount <
Constants.MAX_RETRY_ATTEMPTS }
        failedMessages.clear()

        toRetry.forEach { failed ->
            executor.execute {
                try {
                    val receiver = SmsReceiver()
                    val channel = failed.toChannel()
                    val success = receiver.sendToWebhook(failed.channelTarget,
failed.sender, failed.content, failed.receiverPhoneNumber, channel.type,
failed.showSenderPhone, failed.highlightVerificationCode)
                    if (success) {
                        LogStore.append(context, "重试转发成功 ->
${failed.channelName}")
                    } else {
                        if (failed.retryCount + 1 < Constants.MAX_RETRY_ATTEMPTS) {
                            failedMessages.add(failed.copy(retryCount =
failed.retryCount + 1))
                        } else {
                            LogStore.append(context, "重试转发失败（已达最大次数）->
${failed.channelName}")
                        }
                    }
                } catch (e: Exception) {
                    Log.e(TAG, "retry failed", e)
                }
            }
        }
        saveFailedMessages(context)
    }
}

override fun onReceive(context: Context, intent: Intent) {
    if (intent.action != Telephony.Sms.Intents.SMS_RECEIVED_ACTION) return

    val prefs = context.getSharedPreferences(Constants.PREFS_NAME, Context.MODE_PRIVATE)
    val isEnabled = prefs.getBoolean(Constants.PREF_ENABLED, false)
    if (!isEnabled) return

    // 读取配置项
    val showReceiverPhone = prefs.getBoolean(Constants.PREF_SHOW_RECEIVER_PHONE, true)
    val showSenderPhone = prefs.getBoolean(Constants.PREF_SHOW_SENDER_PHONE, true)

```

```

        val highlightVerificationCode =
prefs.getBoolean(Constants.PREF_HIGHLIGHT_VERIFICATION_CODE, true)

        val channels = loadChannels(prefs)
        val configs = loadConfigs(prefs)

        if (channels.isEmpty() || configs.isEmpty()) {
            LogStore.append(context, "未配置通道或关键词规则，已跳过转发")
            return
        }

        val messages = Telephony.Sms.Intents.getMessagesFromIntent(intent)
        val sb = StringBuilder()
        var sender = ""
        var subscriptionId: Int? = null

        // 尝试从 intent 中获取 subscriptionId
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP_MR1) {
            subscriptionId = intent.getIntExtra("subscription", -1)
            if (subscriptionId == -1) {
                subscriptionId = intent.getIntExtra("slot", -1)
                if (subscriptionId != -1) {
                    // slot 转换为 subscriptionId
                    try {
                        if (ContextCompat.checkSelfPermission(context,
Manifest.permission.READ_PHONE_STATE) == PackageManager.PERMISSION_GRANTED) {
                            val subscriptionManager = SubscriptionManager.from(context)
                            val activeSubscriptions =
subscriptionManager.activeSubscriptionInfoList
                            if (activeSubscriptions != null && activeSubscriptions.size >
subscriptionId) {
                                subscriptionId =
activeSubscriptions[subscriptionId].subscriptionId
                            }
                        }
                    } catch (e: Exception) {
                        Log.e(TAG, "转换 slot 为 subscriptionId 失败", e)
                        subscriptionId = null
                    }
                } else {
                    subscriptionId = null
                }
            }
        }

        for (sms in messages) {
            sender = sms.displayOriginatingAddress ?: sender
            sb.append(sms.displayMessageBody)
        }
        val fullMessage = normalizeContent(sb.toString())

        // 获取接收短信的本机号码

```

```

        val receiverPhoneNumber = if (showReceiverPhone) getReceiverPhoneNumber(context,
subscriptionId) else null

// 消息去重检查
val messageKey = "${sender}_${fullMessage.hashCode()}"
val now = System.currentTimeMillis()
synchronized(recentMessages) {
    cleanupRecentMessages()
    val lastTime = recentMessages[messageKey]
    if (lastTime != null && (now - lastTime) < Constants.DUPLICATE_WINDOW_MS) {
        Log.d(TAG, "跳过重复消息: sender=$sender")
        return
    }
    recentMessages[messageKey] = now
}

// 收集所有匹配项
val matched = mutableListOf<Pair<Channel, KeywordConfig>>()
configs.forEach { cfg ->
    val kw = cfg.keyword.trim()
    val match = if (kw.isEmpty()) true else fullMessage.contains(kw, ignoreCase =
true)
    if (match) {
        val ch = channels.find { it.id == cfg.channelId }
        if (ch != null) matched.add(Pair(ch, cfg))
    }
}

if (matched.isEmpty()) return

// 加载持久化的失败消息
loadFailedMessages(context)

val pendingResult = goAsync()

// 并行发送
executor.execute {
    val latch = java.util.concurrent.CountDownLatch(matched.size)
    try {
        matched.forEach { (ch, cfg) ->
            executor.execute {
                try {
                    if (!isValidUrl(ch.target)) {
                        LogStore.append(context, "通道 ${ch.name} webhook 格式无效:
${ch.target}")
                    } else {
                        var attempt = 0
                        var success = false
                        var backoff = 0L
                        while (attempt < Constants.MAX_RETRY_ATTEMPTS && !success) {
                            if (backoff > 0) {
                                try { Thread.sleep(backoff) } catch (_:
InterruptedException) { }

```

```

        }
        try {
            success = sendToWebhook(ch.target, sender,
fullMessage, receiverPhoneNumber, ch.type, showSenderPhone, highlightVerificationCode)
        } catch (e: Exception) {
            Log.e(TAG, "send attempt ${attempt+1} failed to
${ch.target}", e)
        }
        attempt++
        if (!success) backoff =
Constants.INITIAL_RETRY_BACKOFF_MS * attempt
    }
    if (success) {
        LogStore.append(context, "转发成功 - 来自: $sender ->
${ch.name} (规则: ${cfg.keyword})")
    } else {
        LogStore.append(context, "转发失败 - 来自: $sender ->
${ch.name} (规则: ${cfg.keyword})")
        // 添加到失败队列等待网络恢复时重试
        synchronized(failedMessageLock) {
            if (failedMessages.size <
Constants.MAX_FAILED_MESSAGES) {
                failedMessages.add(FailedMessage.fromChannel(ch,
sender, fullMessage, receiverPhoneNumber, showSenderPhone, highlightVerificationCode, now))
            }
        }
    }
}
} finally {
    latch.countDown()
}
}

val completed = try {
    latch.await(Constants.BROADCAST_TIMEOUT_SECONDS, TimeUnit.SECONDS)
} catch (e: InterruptedException) {
    Log.w(TAG, "await interrupted", e)
    false
}
if (!completed) {
    LogStore.append(context, "部分转发任务超时（等待
${Constants.BROADCAST_TIMEOUT_SECONDS}s 后返回）")
}

// 保存失败消息
saveFailedMessages(context)
} catch (t: Throwable) {
    Log.e(TAG, "unexpected error in parallel send worker", t)
} finally {
    pendingResult.finish()
}
}

```

```

}

// 归一化: 删除 CR, 折叠连续空行为单个换行, trim 首尾空白
private fun normalizeContent(s: String): String {
    return s.replace("\r", "")
        .replace(Regex("\n{2,}"), "\n")
        .trim()
}

/**
 * 获取接收短信的本机号码
 * @param subscriptionId SIM 卡的 subscriptionId, 用于确定是哪个 SIM 卡接收的短信
 */
private fun getReceiverPhoneNumber(context: Context, subscriptionId: Int?): String? {
    try {
        val prefs = context.getSharedPreferences(Constants.PREFS_NAME,
Context.MODE_PRIVATE)

        var simSlotIndex = 1 // 默认假设是 SIM1
        var foundMatchingSim = false

        // 根据 subscriptionId 确定 SIM 卡槽位置
        if (subscriptionId != null && Build.VERSION.SDK_INT >=
Build.VERSION_CODES.LOLLIPOP_MR1) {
            try {
                if (ContextCompat.checkSelfPermission(context,
Manifest.permission.READ_PHONE_STATE) == PackageManager.PERMISSION_GRANTED) {
                    val subscriptionManager = SubscriptionManager.from(context)
                    val activeSubscriptions =
subscriptionManager.activeSubscriptionInfoList
                        activeSubscriptions?.forEachIndexed { index, subInfo ->
                            if (subInfo.subscriptionId == subscriptionId) {
                                simSlotIndex = index + 1 // slot 从 1 开始
                                foundMatchingSim = true
                            }
                        }
                }
            } catch (e: Exception) {
                Log.e(TAG, "确定 SIM 卡槽位置失败", e)
            }
        }

        // 根据 SIM 卡槽位置返回对应的自定义号码
        if (simSlotIndex == 1) {
            val customSim1Phone = prefs.getString(Constants.PREF_CUSTOM_SIM1_PHONE,
null)

            if (!customSim1Phone.isNullOrEmpty()) {
                return customSim1Phone
            }
        } else if (simSlotIndex == 2) {
            val customSim2Phone = prefs.getString(Constants.PREF_CUSTOM_SIM2_PHONE,
null)

            if (!customSim2Phone.isNullOrEmpty()) {

```

```

        return customSim2Phone
    }
}

// 如果没有自定义号码, 但找到了匹配的 SIM, 尝试自动获取
if (ContextCompat.checkSelfPermission(context,
Manifest.permission.READ_PHONE_STATE) == PackageManager.PERMISSION_GRANTED) {
    // 如果有 subscriptionId, 尝试通过 subscriptionId 获取号码
    if (subscriptionId != null && Build.VERSION.SDK_INT >=
Build.VERSION_CODES.LOLLIPOP_MR1 && foundMatchingSim) {
        try {
            val subscriptionManager = SubscriptionManager.from(context)
            val subInfo =
subscriptionManager.getActiveSubscriptionInfo(subscriptionId)
            if (subInfo != null && !subInfo.number.isNullOrBlank()) {
                return subInfo.number
            }
        } catch (e: Exception) {
            Log.e(TAG, "通过 subscriptionId 获取号码失败", e)
        }
    }

    // 回退到默认的获取方式
    val telephonyManager = context.getSystemService(Context.TELEPHONY_SERVICE)
as TelephonyManager
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
        val number = telephonyManager.line1Number
        if (!number.isNullOrBlank()) {
            return number
        }
    } else {
        @Suppress("DEPRECATION")
        val number = telephonyManager.line1Number
        if (!number.isNullOrBlank()) {
            return number
        }
    }
} else {
    Log.d(TAG, "没有 READ_PHONE_STATE 权限, 无法自动获取本机号码")
}
} catch (e: Exception) {
    Log.e(TAG, "获取本机号码失败", e)
}
return null
}

internal fun sendToWebhook(webhookUrl: String, sender: String, content: String,
receiverPhoneNumber: String?, type: ChannelType, showSenderPhone: Boolean,
highlightVerificationCode: Boolean): Boolean {
    val json = when (type) {
        ChannelType.FEISHU -> buildFeishuMessage(sender, content, receiverPhoneNumber,
showSenderPhone, highlightVerificationCode)
    }
}

```

```

        ChannelType.WECHAT -> buildwechatMessage(sender, content, receiverPhoneNumber,
showSenderPhone, highlightVerificationCode)
        ChannelType.DINGTALK -> buildDingtalkMessage(sender, content,
receiverPhoneNumber, showSenderPhone, highlightVerificationCode)
        ChannelType.GENERIC_WEBHOOK -> buildGenericMessage(sender, content,
receiverPhoneNumber, showSenderPhone, highlightVerificationCode)
    }

    val body = json.toString().toRequestBody("application/json; charset=utf-
8".toMediaType())
    val req = Request.Builder()
        .url(webhookUrl)
        .post(body)
        .build()

    client.newCall(req).execute().use { resp ->
        return resp.isSuccessful
    }
}

/**
 * 从短信内容中提取验证码
 * 匹配常见验证码格式：4-8位数字，可能带有"验证码"、"校验码"等关键词
 */
private fun extractVerificationCode(content: String): String? {
    // 常见的验证码关键词
    val keywords = listOf("验证码", "校验码", "动态码", "验证 code", "verification code",
"verify code")
    val hasKeyword = keywords.any { content.contains(it, ignoreCase = true) }

    // 优先匹配：关键词后紧跟的 4-8 位数字
    if (hasKeyword) {
        // 模式1：关键词后面直接跟数字（如"验证码是123456"）
        val pattern1 = Regex("(?:验证码|校验码|动态码|验证|verification|verify)[^\\d]*(\\d{4,8})"", RegexOptions.IGNORE_CASE)
        pattern1.find(content)?.let { return it.groupValues[1] }

        // 模式2：关键词附近的数字（关键词后20字符内）
        val pattern2 = Regex("(?:验证码|校验码|动态码|验证|verification|verify){0,30}? (\\d{4,8})"", RegexOptions.IGNORE_CASE)
        pattern2.find(content)?.let { return it.groupValues[1] }
    }

    // 匹配独立的4-8位数字（作为备选）
    val pattern3 = Regex("(?:\\b(\\d{4,8})\\b)""")
    val matches = pattern3.findAll(content).map { it.groupValues[1] }.toList()

    // 如果有多个匹配，返回最长的那个（更可能是验证码）
    if (matches.isNotEmpty()) {
        return matches.maxByOrNull { it.length }
    }

    return null
}

```



```

}

/**
 * 构建消息内容，突出显示验证码
 */
private fun buildMessageWithHighlightedCode(sender: String, content: String,
receiverPhoneNumber: String?, showSenderPhone: Boolean, highlightVerificationCode: Boolean):
String {
    val parts = mutableListOf<String>()
    val code = if (highlightVerificationCode) extractVerificationCode(content) else null

    if (code != null) {
        parts.add("验证码: $code")
    }
    if (receiverPhoneNumber != null) {
        parts.add("本机: $receiverPhoneNumber")
    }
    if (showSenderPhone) {
        parts.add("来自: $sender")
    }
    parts.add(content)

    return parts.joinToString("\n")
}

private fun buildwechatMessage(sender: String, content: String, receiverPhoneNumber:
String?, showSenderPhone: Boolean, highlightVerificationCode: Boolean): JSONObject {
    val json = JSONObject()
    json.put("msgtype", "text")
    val text = JSONObject()
    text.put("content", buildMessageWithHighlightedCode(sender, content,
receiverPhoneNumber, showSenderPhone, highlightVerificationCode))
    json.put("text", text)
    return json
}

private fun buildDingtalkMessage(sender: String, content: String, receiverPhoneNumber:
String?, showSenderPhone: Boolean, highlightVerificationCode: Boolean): JSONObject {
    val json = JSONObject()
    json.put("msgtype", "text")
    val text = JSONObject()
    text.put("content", buildMessageWithHighlightedCode(sender, content,
receiverPhoneNumber, showSenderPhone, highlightVerificationCode))
    json.put("text", text)
    return json
}

private fun buildFeishuMessage(sender: String, content: String, receiverPhoneNumber:
String?, showSenderPhone: Boolean, highlightVerificationCode: Boolean): JSONObject {
    val json = JSONObject()
    json.put("msg_type", "text")
    val text = JSONObject()

```

```

        text.put("text", buildMessageWithHighlightedCode(sender, content,
receiverPhoneNumber, showSenderPhone, highlightVerificationCode))
        json.put("content", text)
        return json
    }

    private fun buildGenericMessage(sender: String, content: String, receiverPhoneNumber:
String?, showSenderPhone: Boolean, highlightVerificationCode: Boolean): JSONObject {
        val json = JSONObject()
        if (showSenderPhone) {
            json.put("sender", sender)
        }
        json.put("receiver", receiverPhoneNumber)
        json.put("content", content)
        if (highlightVerificationCode) {
            json.put("verificationCode", extractVerificationCode(content))
        }
        json.put("timestamp", System.currentTimeMillis())
        return json
    }

    private fun isValidUrl(s: String): Boolean {
        return try {
            val url = URL(s)
            (url.protocol == "http" || url.protocol == "https") && url.host.isNotBlank()
        } catch (e: MalformedURLException) {
            false
        }
    }

    private fun loadChannels(prefs: android.content.SharedPreferences): List<Channel> {
        val arrStr = prefs.getString(Constants.PREF_CHANNELS, "[]") ?: "[]"
        return try {
            val arr = org.json.JSONArray(arrStr)
            (0 until arr.length()).map { i ->
                val o = arr.getJSONObject(i)
                val typeStr = o.optString("type", "WECHAT")
                val type = try { ChannelType.valueOf(typeStr) } catch (t: Throwable) {
ChannelType.WECHAT }
                Channel(o.getString("id"), o.getString("name"), type, o.getString("target"))
            }
        } catch (t: Throwable) {
            emptyList()
        }
    }

    private fun loadConfigs(prefs: android.content.SharedPreferences): List<KeywordConfig> {
        val arrStr = prefs.getString(Constants.PREF_KEYWORD_CONFIGS, "[]") ?: "[]"
        return try {
            val arr = org.json.JSONArray(arrStr)
            (0 until arr.length()).map { i ->
                val o = arr.getJSONObject(i)

```

```
        KeywordConfig(o.getString("id"), o.getString("keyword"),  
o.getString("channelId"))  
    }  
    } catch (t: Throwable) {  
        emptyList()  
    }  
}  
}
```

说明

本文件包含短信转发助手（V2.7.2）的全部源程序鉴别材料，所有文件均包含完整的版权信息。

随着软件升级，本源代码会有轻微变化。

著作权人：华昊科技有限公司

开发者：王士辉

版本：V2.7.2

联系邮箱：huahao@email.cn

